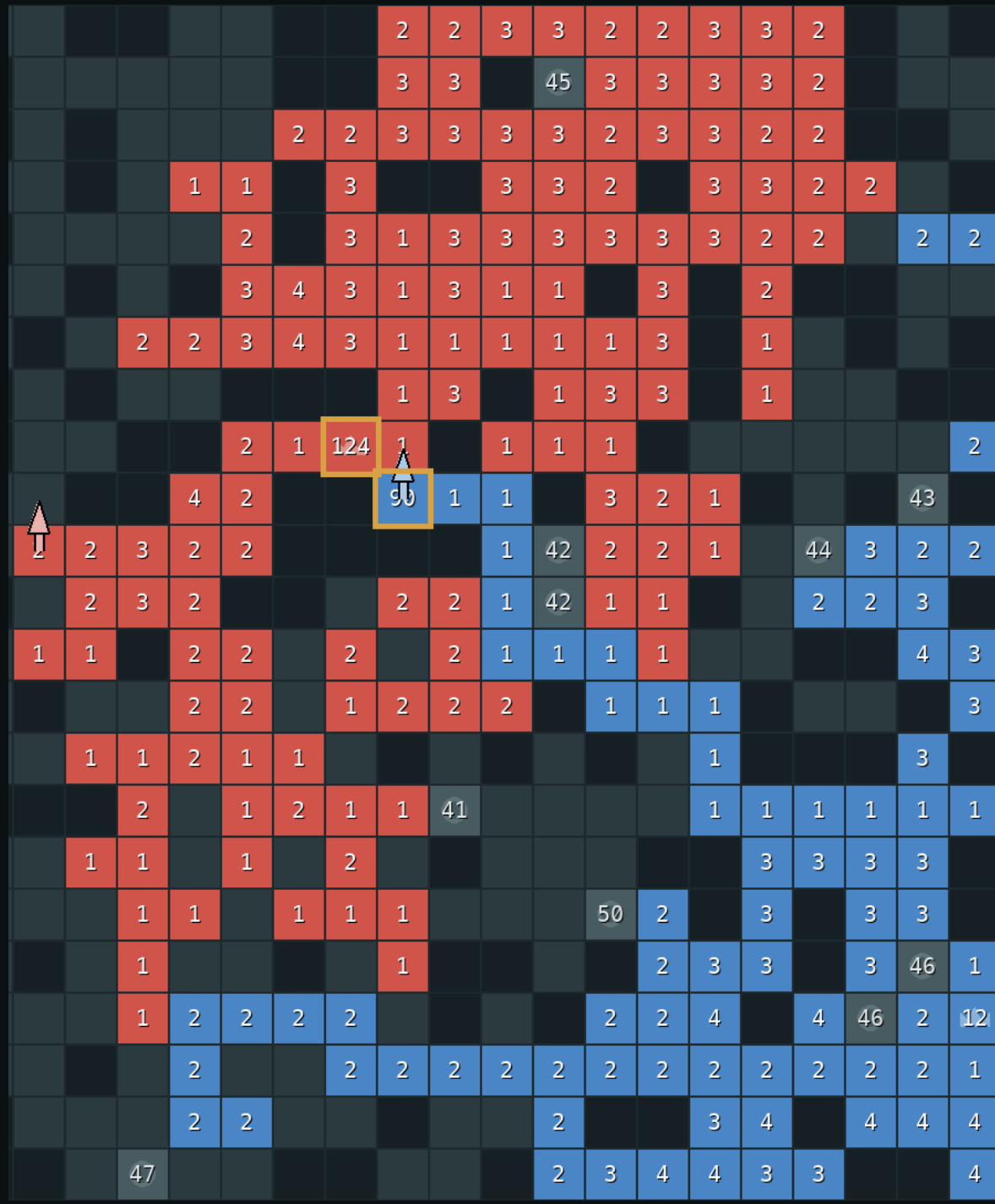
DAC-203 · ARTIFICIAL INTELLIGENCE · IIT ROORKEE

How our bot plays Generals

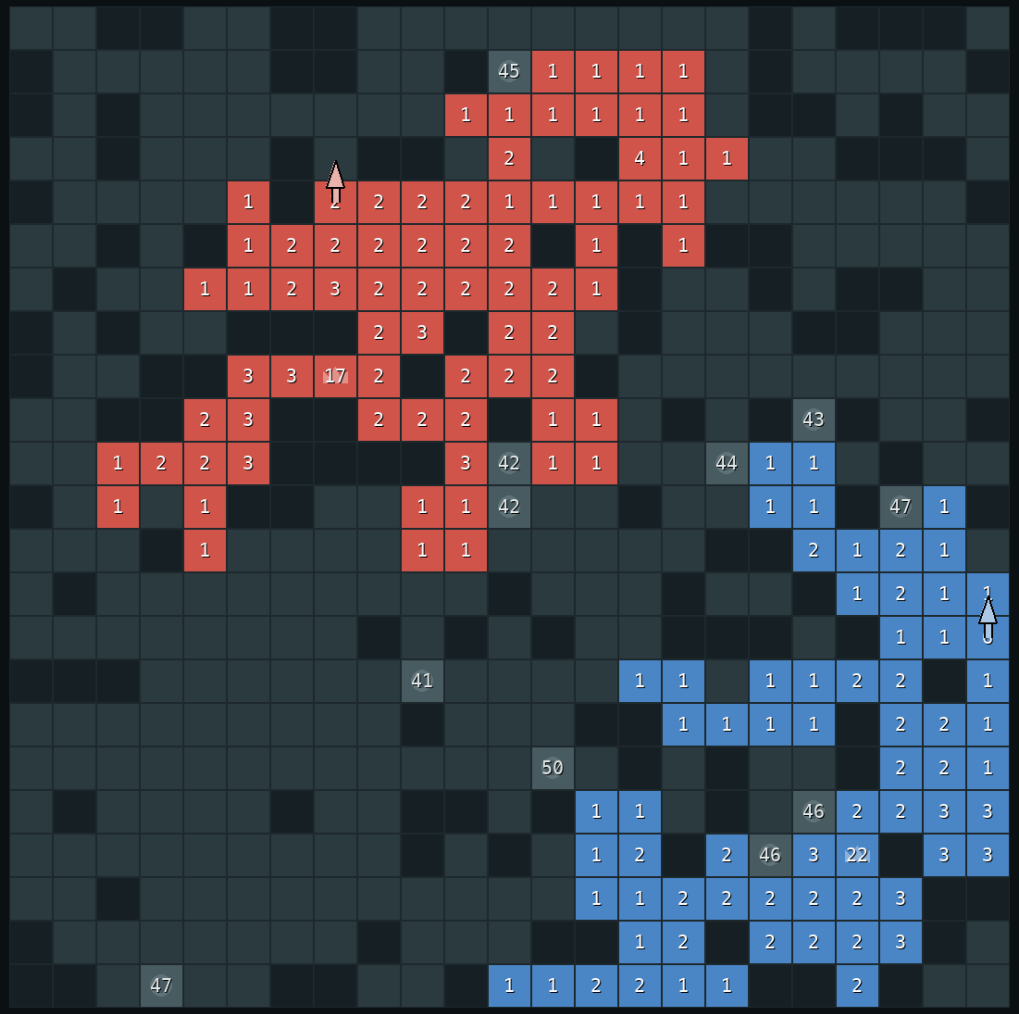
v51: a game agent built from the textbook and six research papers.



Right: a real moment from a game. Our bot is red.

QUICK RECAP

The game in thirty seconds



Numbers are soldiers

Red is our bot, blue is the opponent.

Lose your general, lose the game

The crown marks each player's general.

One move per turn, both at once

Move one stack one square. Fights subtract.

Soldiers grow on their own

Generals and castles fast, land slowly.

THE KEY IDEA

Walking costs soldiers, waiting earns them

start



8 turns later



30 → 22

the attacker leaves a soldier on every square

12 → 16

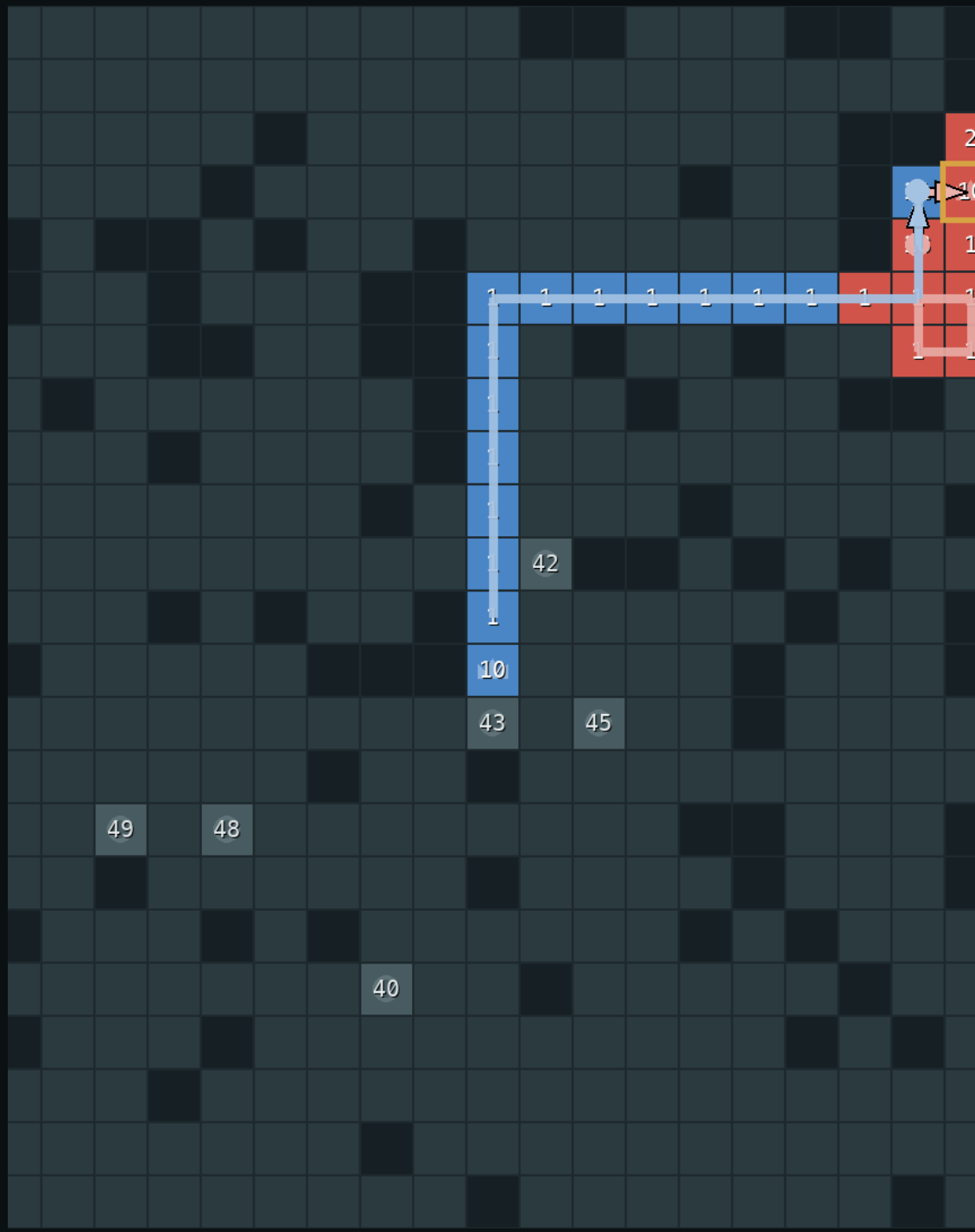
the general grows while it waits

A general falls only when its soldiers have left, or when several stacks hit it one after another.

1

How we got here

Fifty versions, three changes of direction. What the five papers from our first two presentations gave us, and the games that made us change course.



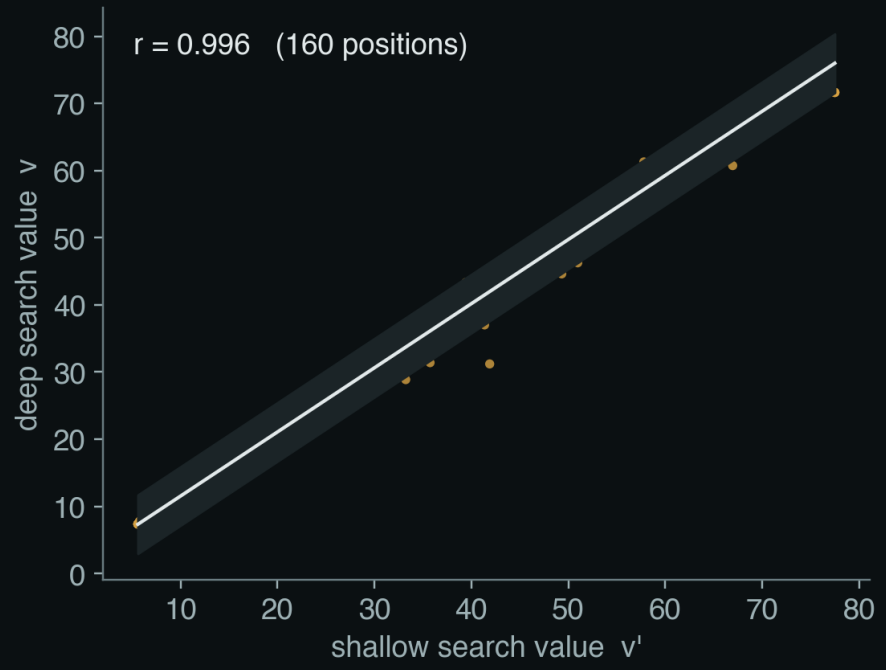
The five papers we started with

Paper	What it is for	What it assumes	What we measured
ARA* Likhachev et al. 2003	find a path fast, improve it while time is left	finding the path is the hard part	a full distance map takes 0.04 ms
RTAA* Koenig & Likhachev 2006	learn better distance estimates while moving	distances are unknown	the map is visible: distances are exact
Depression avoidance Hernández & Baier 2012	escape dead ends of a wrong estimate	estimates can be wrong	exact distances have no dead ends
ProbCut Buro 1995	search deeper by skipping hopeless branches	deeper search wins more	deeper search won no more games
Metareasoning Russell & Wefald 1991	decide how long to think	thinking time is scarce	we use 10 ms of the 500 ms allowed

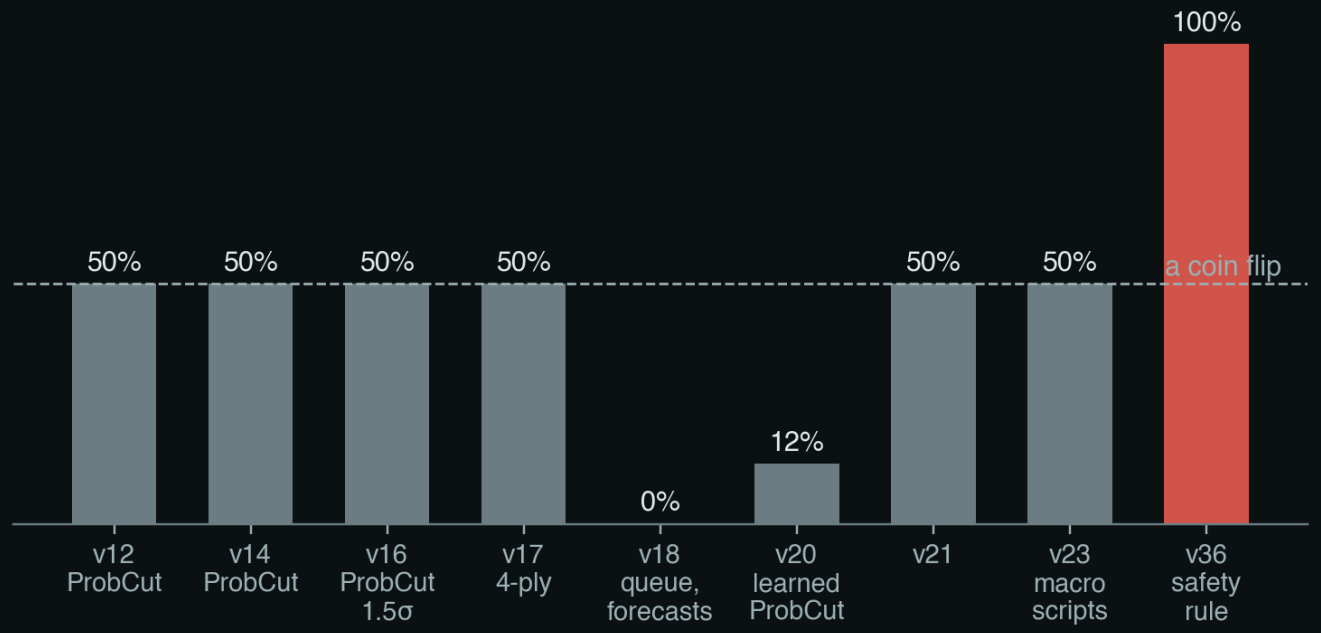
All five make search faster or deeper. Our games were lost for another reason: a general left without enough soldiers.

Evidence 1: searching deeper didn't help

$$v \approx a\,v' + b + e, \quad e \sim \mathcal{N}(0, \sigma^2); \qquad \text{cut if } \frac{a\,v' + b - \beta}{\sigma} \geq t$$

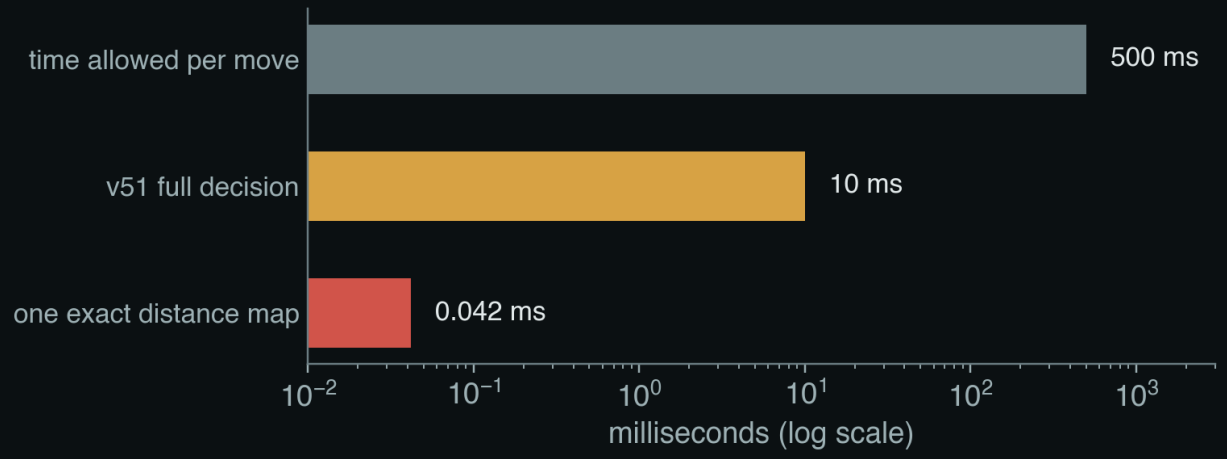


Shallow predicts deep almost exactly. 160 positions; v = 0.955 v′ + 1.97, σ = 2.18.



Win rate against v11, our best bot at the time. Search-paper bots (grey) stay at a coin flip or below. v36 (red) is the first bot built on the safety rule.

Evidence 2: we were never short of time



Time allowed per move: 500 ms. The final bot decides in about 10 ms. An exact distance map over the whole board: 0.04 ms.

TERM **Admissible vs exact heuristic (AIMA ch. 3)**

A heuristic h estimates the distance to the goal. When h equals the true distance h*, search goes straight to the goal and there is nothing left to learn.

RTAA*: raise the estimate of every state it searched

$$h(s) \leftarrow g(\bar{s}) + h(\bar{s}) - g(s)$$

With exact distances the update changes nothing

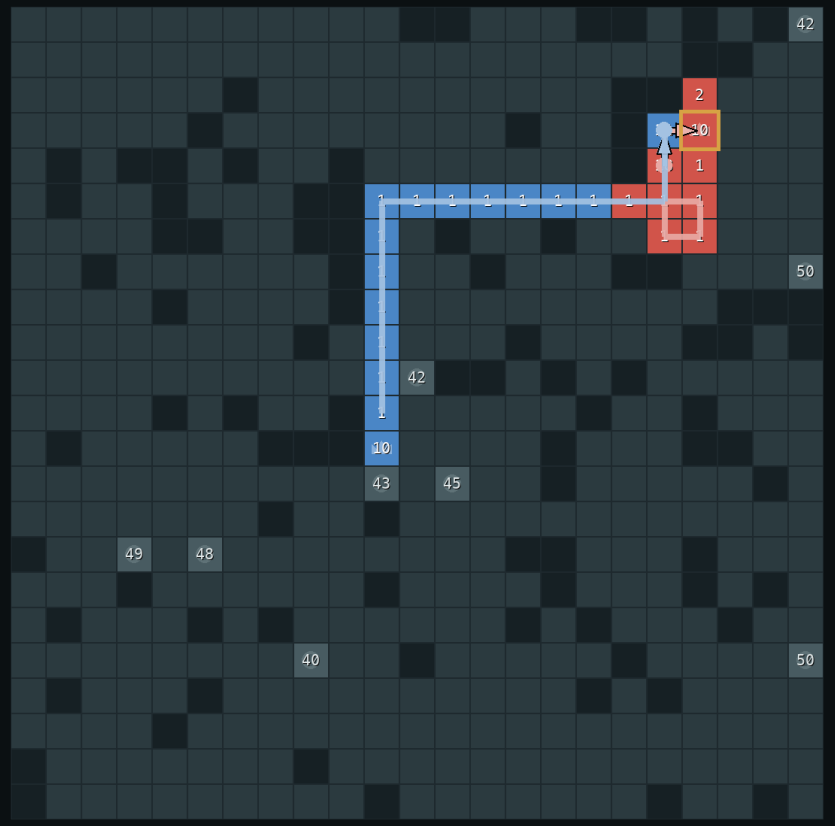
$$h = h^* \;\Rightarrow\; g(\bar{s}) + h^*(\bar{s}) - g(s) \;=\; h^*(s) \qquad \text{(no change)}$$

Metareasoning: think more only if it is worth the time

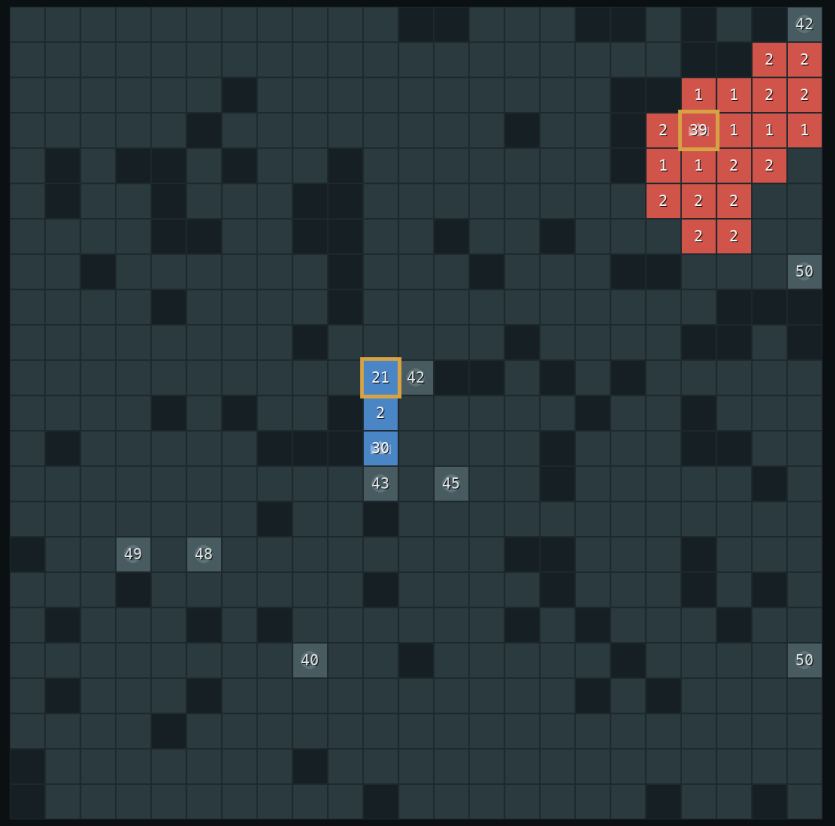
$$\mathrm{VOC}(c) \;=\; \mathbb{E}\,[U \text{ after thinking}] \;-\; \mathbb{E}\,[U \text{ now}] \;-\; \mathrm{cost}(c)$$

Time cost close to zero, and deeper thinking rarely changed the move: the value of extra thinking was close to zero.

Evidence 3: how the search-era bot actually lost



v11, turn 98. At turn 81 it moved 40 soldiers out of its general. Its stack circles (light red); the rusher walks straight in (light blue).



v36, same map, turn 98. 39 soldiers stay in the general. The rusher's 21 can't break it.

25 / 60

games v11 lost its general to two simple rush bots

24 / 25

of those games won by v36 on the same maps

turn 99

v11's general falls: 10 soldiers against 21

Change 1: a safety rule before anything else (v36)

In words: for every enemy stack, my general must still win if that stack walks in.

$$\forall\, s: \quad G + \text{growth}(d_s) \;>\; x_{\text{arrive}}(s)$$

v11's move at turn 81, if the general keeps 1:

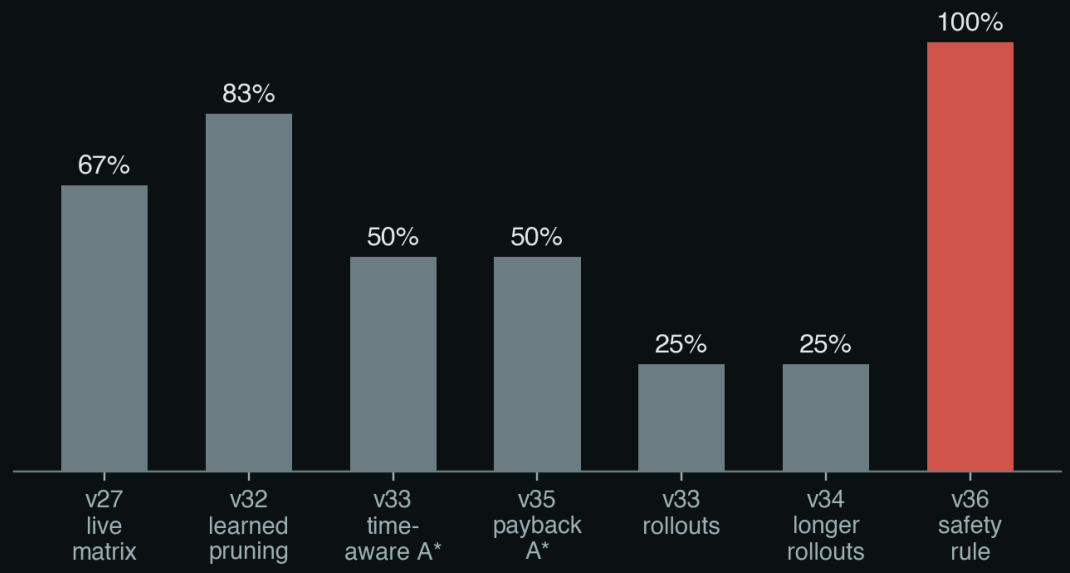
$$1 + \lfloor 18/2 \rfloor = 10 \;<\; 42 - 18 = 24 \quad \Rightarrow \quad \text{refuse}$$

If it keeps its 40:

$$40 + 9 = 49 \;>\; 24 \quad \Rightarrow \quad \text{allowed (but keep the 40 home)}$$

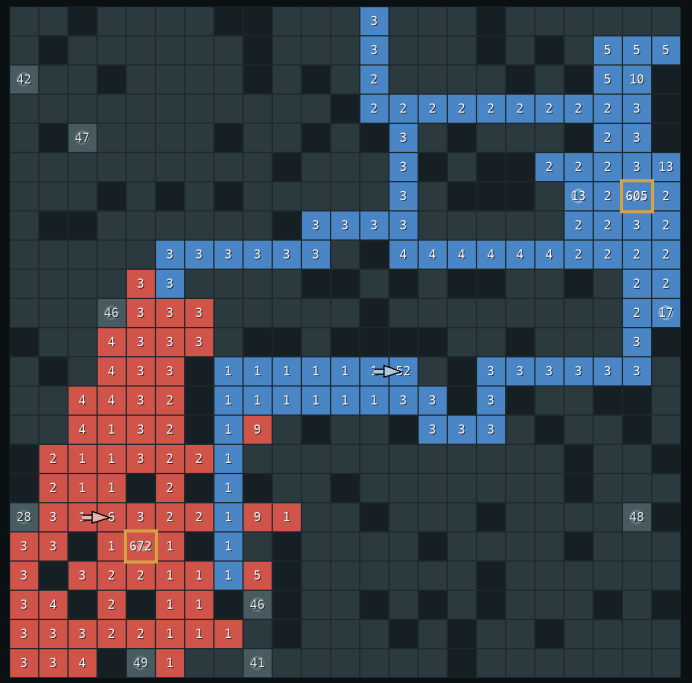
TERM **Constraint before optimisation**

First rule out every move that breaks a hard constraint, then choose the best of what is left. Here the constraint is: the general must survive any single enemy walk.

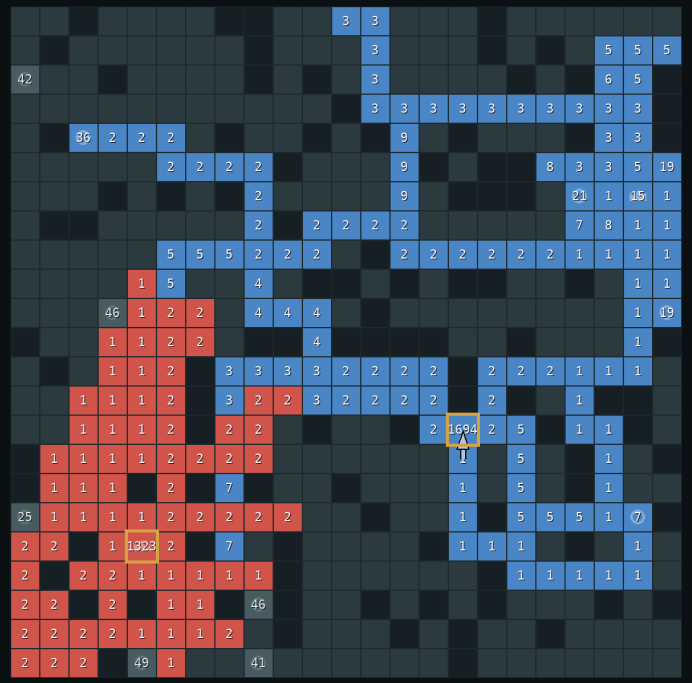


Win rate against the direct rusher. Search-era bots (grey) on 4 to 6 game sets; v36 (red) on 20 games.

Change 2: people beat it by collecting (v42)



Turn 520. Human's biggest stack 605, bot's general 672.



Turn 845. Collected: 1,694 against 1,323.

Here the bot is red, the human blue.

No single human stack was bigger than the bot's general, so the old rule said: safe.

But the human held more land and castles 4 to 0, and collected it all into one stack.

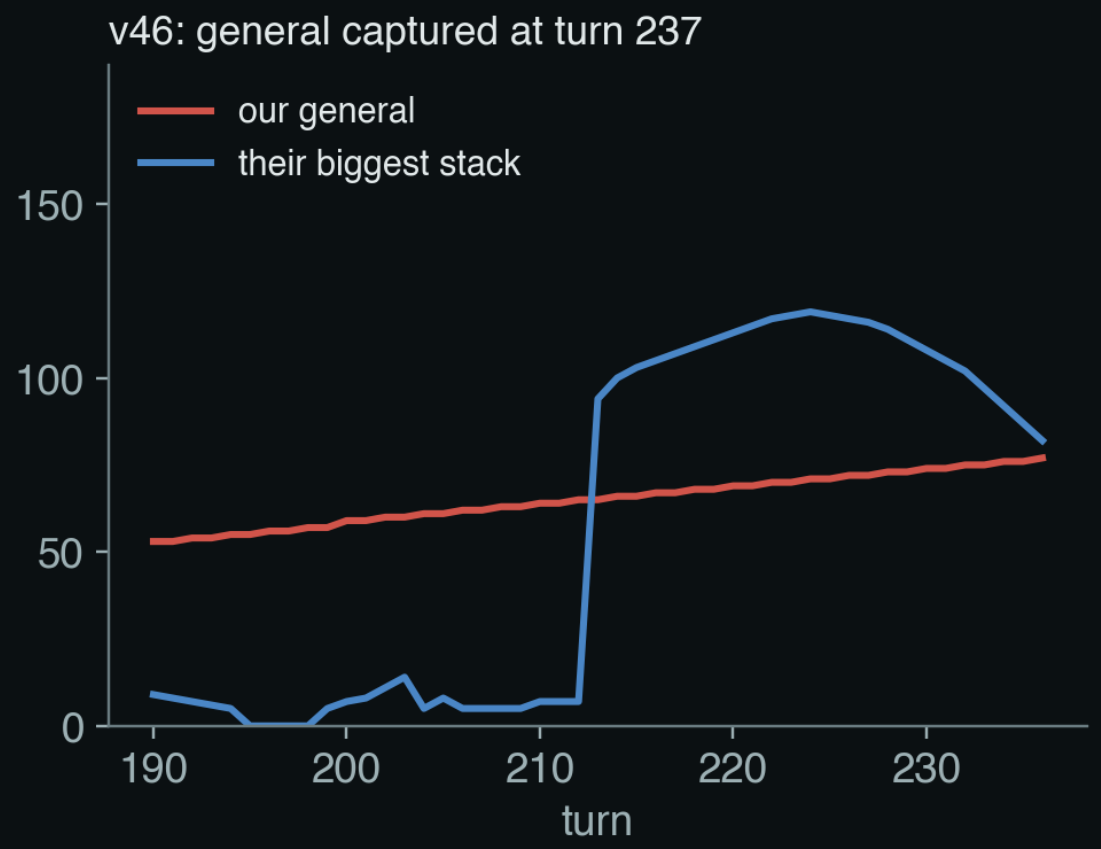
New rule: count everything that can be collected in T moves, on both sides.

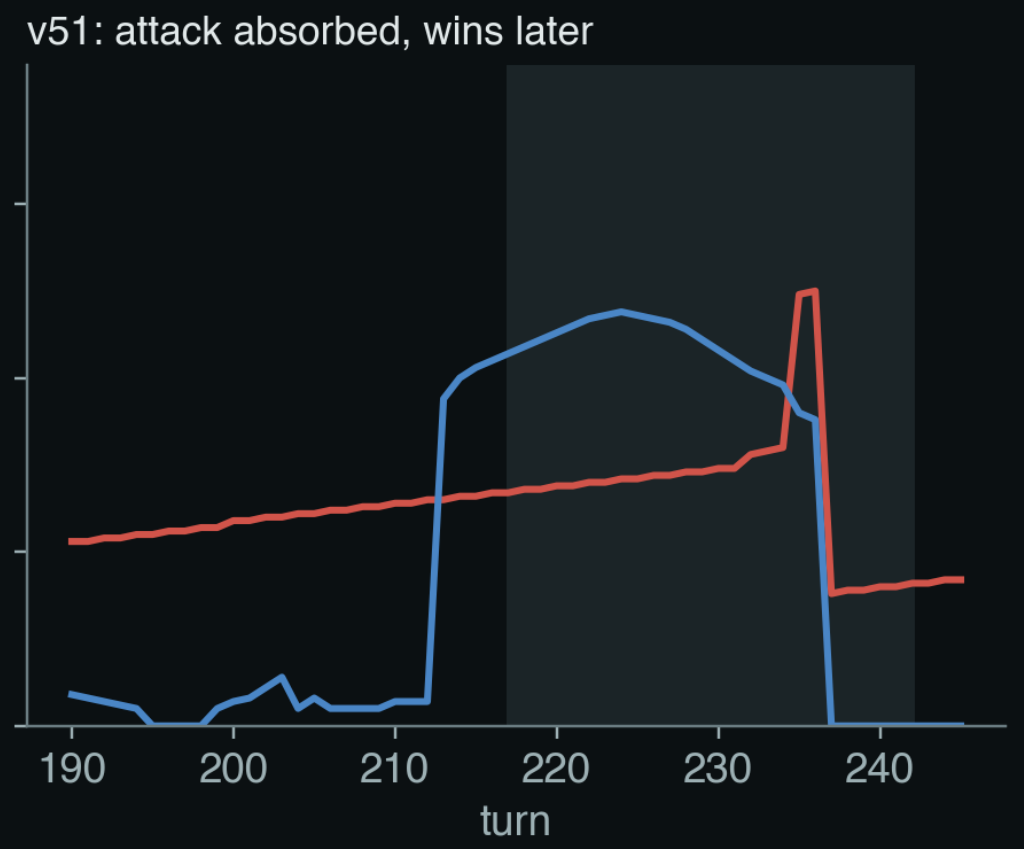
Borrowed from Human.exe (Straka et al. 2026).

Old rule $m_{\text{old}} = \min_{s} \left[G + \text{growth}(d_s) - x_s \right]$ one stack at a time

New rule (idea 2) $m_{\text{new}} = \min_{T} \left[G + \text{growth}(T) + D_{\text{us}}(T) - D_{\text{them}}(T) \right]$

Change 3: defend once, and keep defending (v51)





v46 : $\text{defend}_t \Leftrightarrow m_t \leq \text{pad}$ v51 : defend while incoming and need > 0

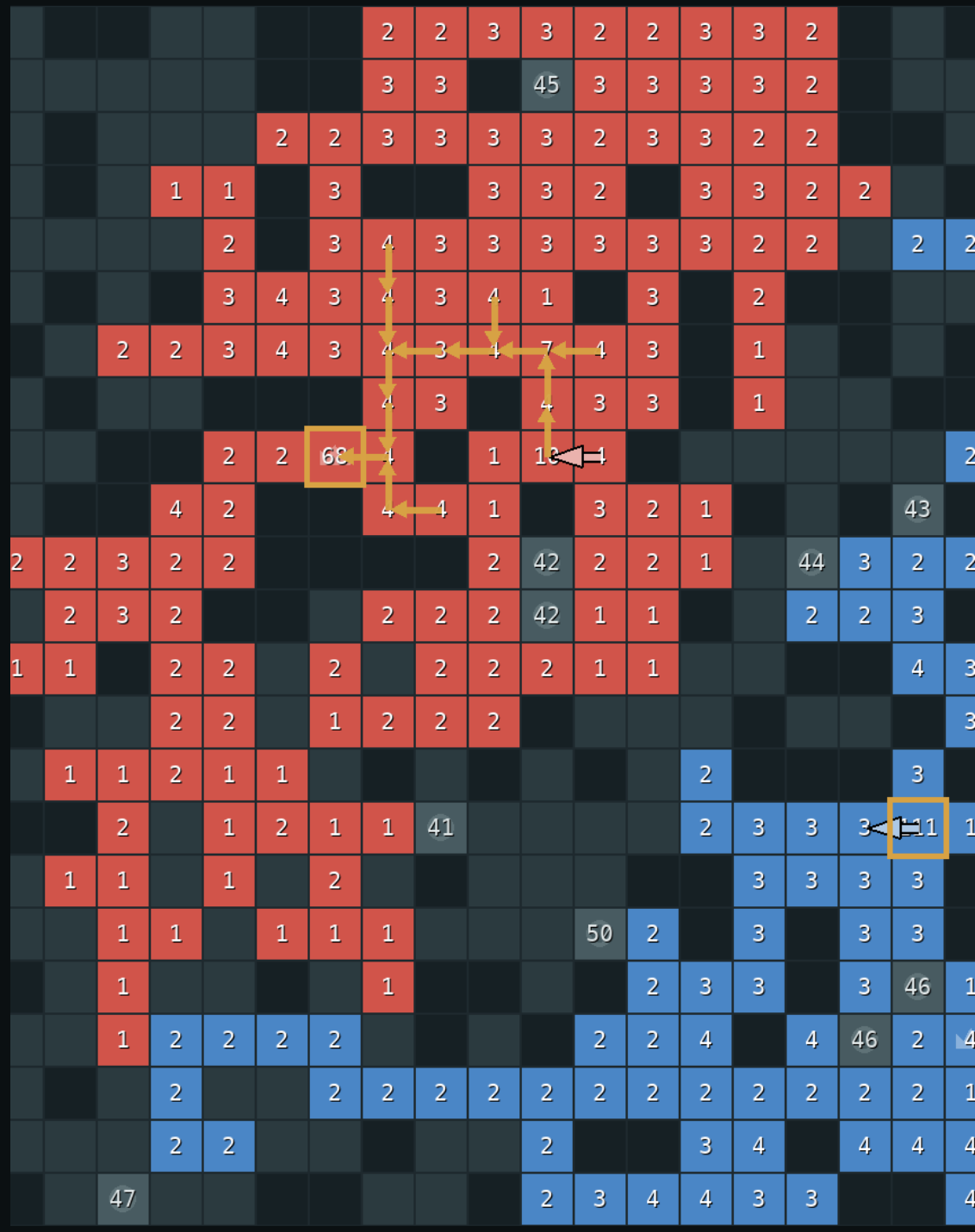
The whole path: every change came from a lost game

Version	What changed	Why
v1–v35	faster, deeper search: the five papers	no better than a coin flip against v11
v36	safety rule: never leave the general beatable	v11 lost 25 of 60 to rush bots
v37	plans, 40-turn lookahead, learned win estimate	v36 stalled: 16 of 20 against land grabbers
v39	never swap places with an attacker	replays of games humans won
v42–v44	count collected armies; reserve; counter-strike	a human collected 1,694 against 1,323
v46	guess the opponent's style	17–3 against v37
v51	commit to defence; trained style classifier	v46 lost the gather-and-charge game

2

How v51 decides a move

Eight ideas. For each: the concept and its paper first, then how v51 uses it.



What happens every turn

1 Look
read the whole board

→

2 Guess
what kind of player is the opponent?

→

3 Check
is an attack coming?

→

4 Decide
defend now, or pick a plan

→

5 Test
could this move lose the general?

→

6 Move
one move, every half second

Two rules never change: a move that fails the test is never played, and when an attack is coming, defence gets every move until it is over.

TERM **Utility-based agent (AIMA ch. 2)**
An agent that picks actions by estimating how good the result will be, instead of following fixed rules.

TERM **Simultaneous-move game (AIMA ch. 18)**
Both players commit before seeing the other's move, so each turn is closer to rock-paper-scissors than to chess.

Updating a belief with evidence

You hold a probability for each explanation. New evidence raises the explanations that predicted it and lowers the others. That is Bayes' rule.

Do it again every turn, keeping what you believed before, and you get a filter: a belief that follows the game.

Bayes' rule: belief in y after seeing evidence e

$$P(y \mid e) = \frac{P(e \mid y)\, P(y)}{P(e)}$$

Repeated every turn: today's belief from yesterday's

$$b_t(y) \propto P(x_t \mid y) \cdot b_{t-1}(y)$$

A classifier turns measurements x into probabilities

$$P(y \mid x) = \frac{e^{z_y}}{\sum_k e^{z_k}}, \qquad z = f_\theta(x)$$

THE PAPER

Synnaeve & Bessière, AIIDE 2011

Predicted a StarCraft player's opening strategy with a Bayesian model updated from what the player had built so far. Weber & Mateas (CIG 2009) trained classifiers on the same kind of features.

EXAMPLE (illustrative) two styles, 50/50; the general empties: 90% likely for a builder, 5% for a land grabber

$$P(\text{builder} \mid e) = \frac{0.9 \times 0.5}{0.9 \times 0.5 + 0.05 \times 0.5} = 0.95$$

IDEA 1

Guess what kind of player you're facing

FROM THE BOOK updating beliefs with probability

FROM THE PAPER Synnaeve & Bessière 2011; Weber & Mateas 2009

The bot can see the whole board, but not the opponent's plan. So it keeps a guess over ten playing styles, like "rushes early" or "builds up, then attacks".

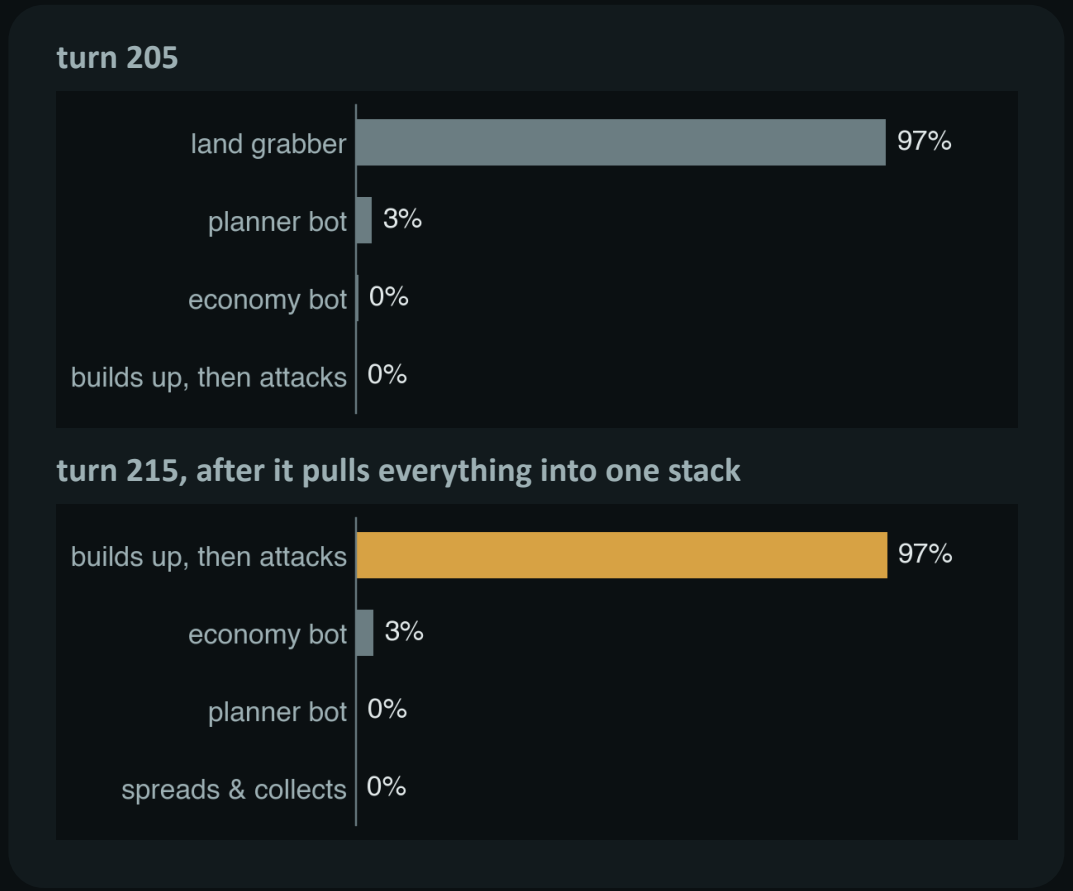
Each turn it measures a few things about the opponent: its land, how much army sits in its general, how big its biggest stack is, and whether that stack is getting closer.

A small neural network turns those measurements into a probability for each style.

TERM **Classifier (AIMA ch. 19, 21)**

A model that sorts an input into one of several categories. Here a small neural network sorts the opponent's stats into 10 styles.

Right about 70% of the time by mid-game on new maps. Random guessing: 10%.



IDEA 1

The guess changes as the game goes on

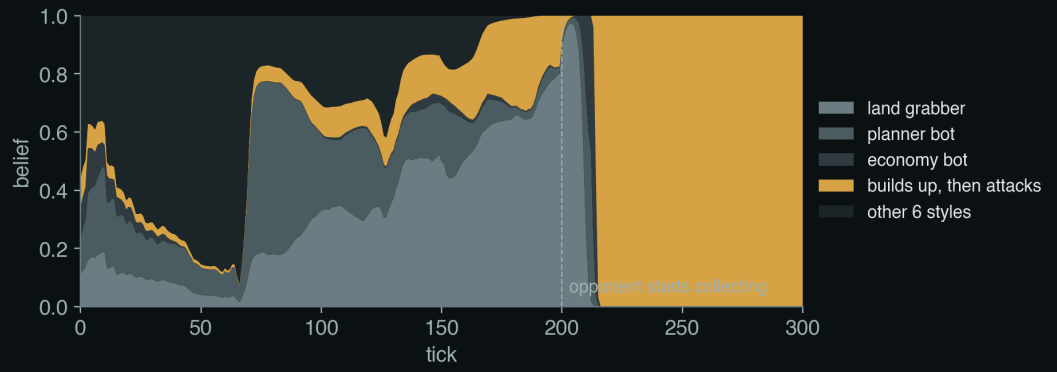
In words: today's guess = mostly yesterday's guess, nudged toward what today's board suggests.

$$\ell_t(y) = 0.95\,\ell_{t-1}(y) + 0.4 \log P(y \mid x_t)$$

ℓ(y) how much evidence we have that the opponent plays style y

P(y | x) what the network thinks after seeing today's board x

0.95 old evidence fades slowly, so a change of style is noticed



Until turn 200 the guess is "land grabber", which is correct: before its attack, this opponent only grabs land. At turn 215 the guess flips to "builds up, then attacks".

TERM **Bayesian filtering (AIMA ch. 14)**

Keep a probability for each possibility and update it with every new observation, using Bayes' rule. Here old evidence also fades slowly.

Packing the most into a limited budget

You have several items, each with a value and a cost, and a fixed budget. Which items do you take? This is the knapsack problem.

Here an item is a stack walking to our general: its value is the soldiers it delivers, its cost is the moves it takes. The budget is the number of turns.

The knapsack problem: take items i to maximise value within budget W

$$\max \sum_i v_i\, x_i \quad \text{such that} \quad \sum_i w_i\, x_i \le W, \qquad x_i \in \{0,1\}$$

Meanwhile the general grows by one every two turns

$$G(t+T) \;=\; G(t) \;+\; \lfloor T/2 \rfloor$$

THE PAPER

Straka, Lisý & Schmid, 2026 (Human.exe)

Describes Human.exe, the strongest hand-written Generals bot. It decides how to gather its army by solving exactly this kind of budgeted collection problem.

EXAMPLE three enemy stacks bring 21, 16 and 15 in 25 moves; our general has 40

$$21 + 16 + 15 = 52 \;\ge\; 40 + \lfloor 25/2 \rfloor = 52$$

IDEA 2

Is my general safe?

FROM THE BOOK searching for shortest paths

FROM THE PAPER Straka, Lisý & Schmid 2026 (Human.exe)

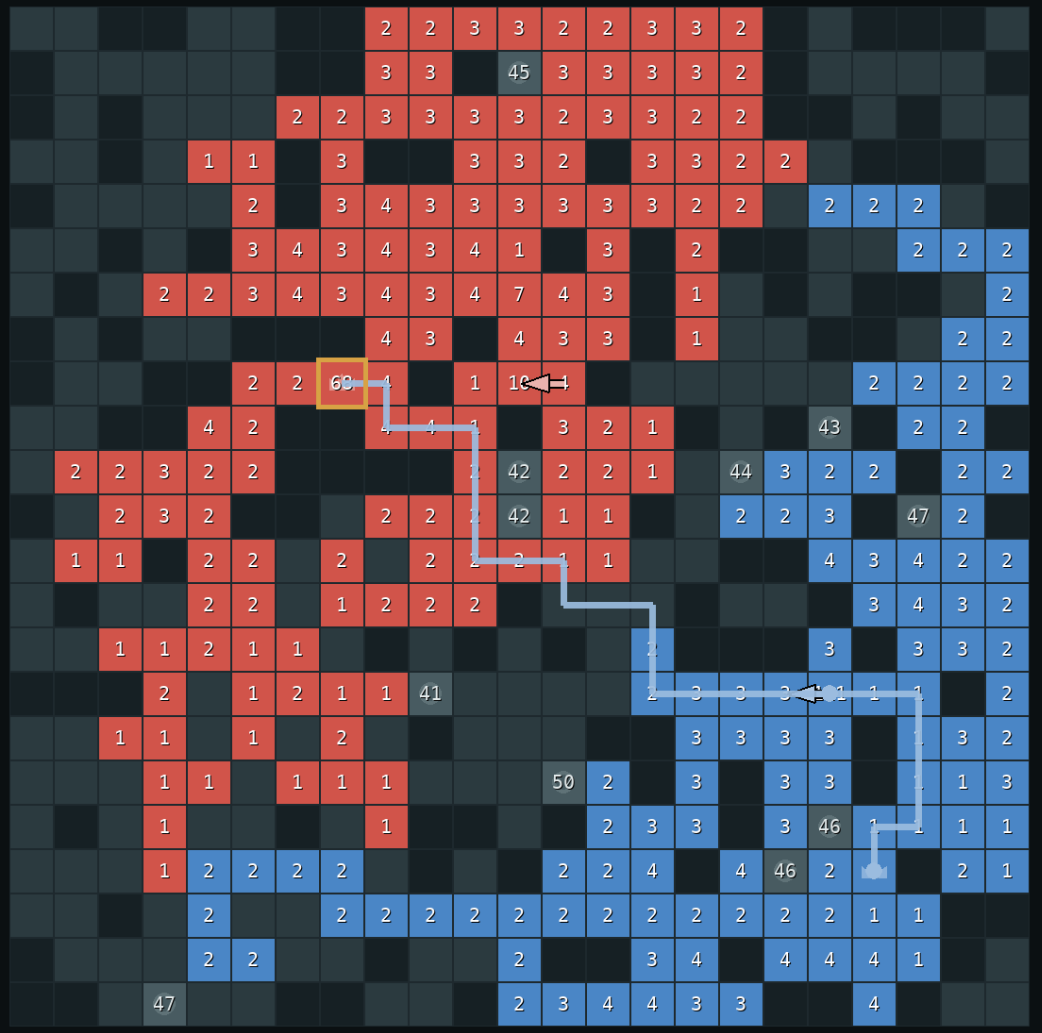
Every turn the bot compares two numbers:

Threat: how many enemy soldiers could reach my general if the enemy started walking its stacks in now.

Defence: how many my general will have by then, plus what my own stacks can bring home in time.

Defence minus threat is the safety score. Small means danger.

Right: the enemy's route (light blue) to our general (gold box). 111 soldiers, 18 squares away.



IDEA 2

The safety score during the game

In words: what my side can have at the general, minus what the enemy can bring, at the worst moment.

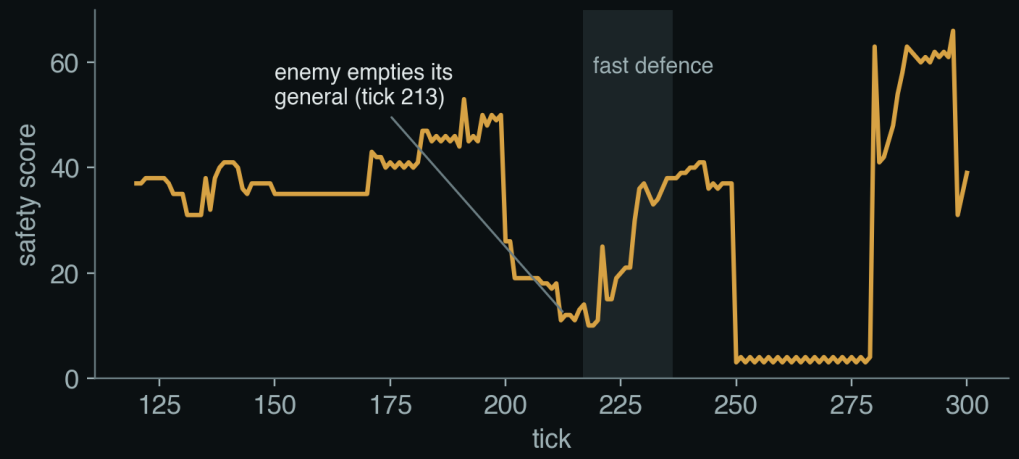
$$m = \min_{T} \left[G + \text{growth}(T) + D_{\text{us}}(T - \text{lead}) - D_{\text{them}}(T) \right]$$

G soldiers on our general now

growth(T) what the general gains in T turns

D_us, D_them soldiers each side can walk to our general in T turns

lead our stacks must arrive a little early



At turn 213 the opponent empties its general into one stack. The score drops from 26 to 12.

TERM **NP-hard, solved greedily**

Picking the best stacks to walk in a limited number of moves is NP-hard: no fast exact method is known. A greedy algorithm takes the biggest stack first, then the next.

IDEA 3 · THE CONCEPT

Planning against every possible reply

Some problems have two kinds of choice. At an OR point we pick one option, and one good option is enough. At an AND point the world picks, and our plan must work for every option.

A move is safe only if it survives every attack the enemy could make. The tree of choices with these two kinds of point is called an AND-OR tree.

A move is accepted only if the general survives every reply b

$$\text{accept}(m) \Leftrightarrow \forall b \in B: \ \text{general survives}(m, b)$$

Scoring the tree: we take the best at OR points, the worst at AND points

$$\text{OR node}: \ \max_{m} \qquad \text{AND node}: \ \min_{b}$$

FROM THE BOOK

Russell & Norvig, AIMA chapter 4

Introduces AND-OR trees for planning when the outcome of an action is not under our control: a plan counts only if it works for every outcome.

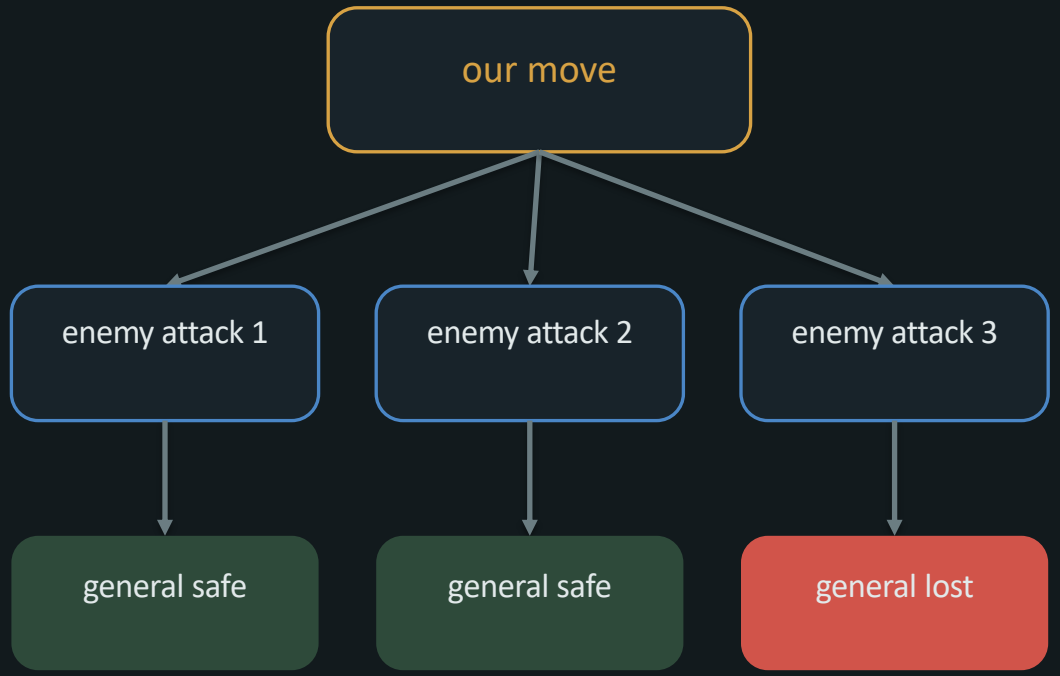
EXAMPLE three possible enemy attacks; the general survives two of them

$$\text{safe} \wedge \text{safe} \wedge \text{lost} = \text{reject}$$

IDEA 3

Test every move before making it

FROM THE BOOK planning against every possible reply (AND-OR search)



One loss is enough to throw the move away.

1. Try the move on a copy of the board. If the general is still safe, play it.

2. If not, imagine the enemy attacking along each likely route, with our bot defending.

3. If the general falls in any of these imagined games, never play the move.

TERM **AND-OR search (AIMA ch. 4)**

A search tree with two kinds of node. At OR nodes we choose one option; at AND nodes every option must work. A move is good only if it survives all enemy attacks.

Collecting as much as possible with a price on effort

We want a tree of routes, all ending in our general, that collects the most soldiers. Every square on a route costs one move.

Trick: put a price λ on each move. Maximise prize minus price. A high price gives a small, quick tree; a low price gives a big, slow one. Search for the price that makes the tree fit the time left.

Prize-collecting Steiner tree: prize p_v on each square, price λ per move

$$\max_{\text{tree} \ni g} \sum_{v \in \text{tree}} p_v \;-\; \lambda \cdot |\text{edges}|$$

Binary search on the price: halve the range each time

$$\lambda \leftarrow \frac{\lambda_{\text{lo}} + \lambda_{\text{hi}}}{2}, \qquad \text{steps} = \left\lceil \log_2 \frac{\text{range}}{\varepsilon} \right\rceil$$

THE PAPER

Straka, Lisý & Schmid, 2026 (Human.exe)

Human.exe gathers its army with this exact structure, a prize-collecting Steiner tree, and searches for the price per move so the plan fits its turn budget.

EXAMPLE prices between 0 and 120, to within 1

$$\text{steps} = \left\lceil \log_2 \frac{120}{1} \right\rceil = \lceil 6.9 \rceil = 7$$

IDEA 4

When an attack is coming, defend fast

FROM THE BOOK turning a goal into a search problem

FROM THE PAPER Straka, Lisý & Schmid 2026 (Human.exe)

1 Notice

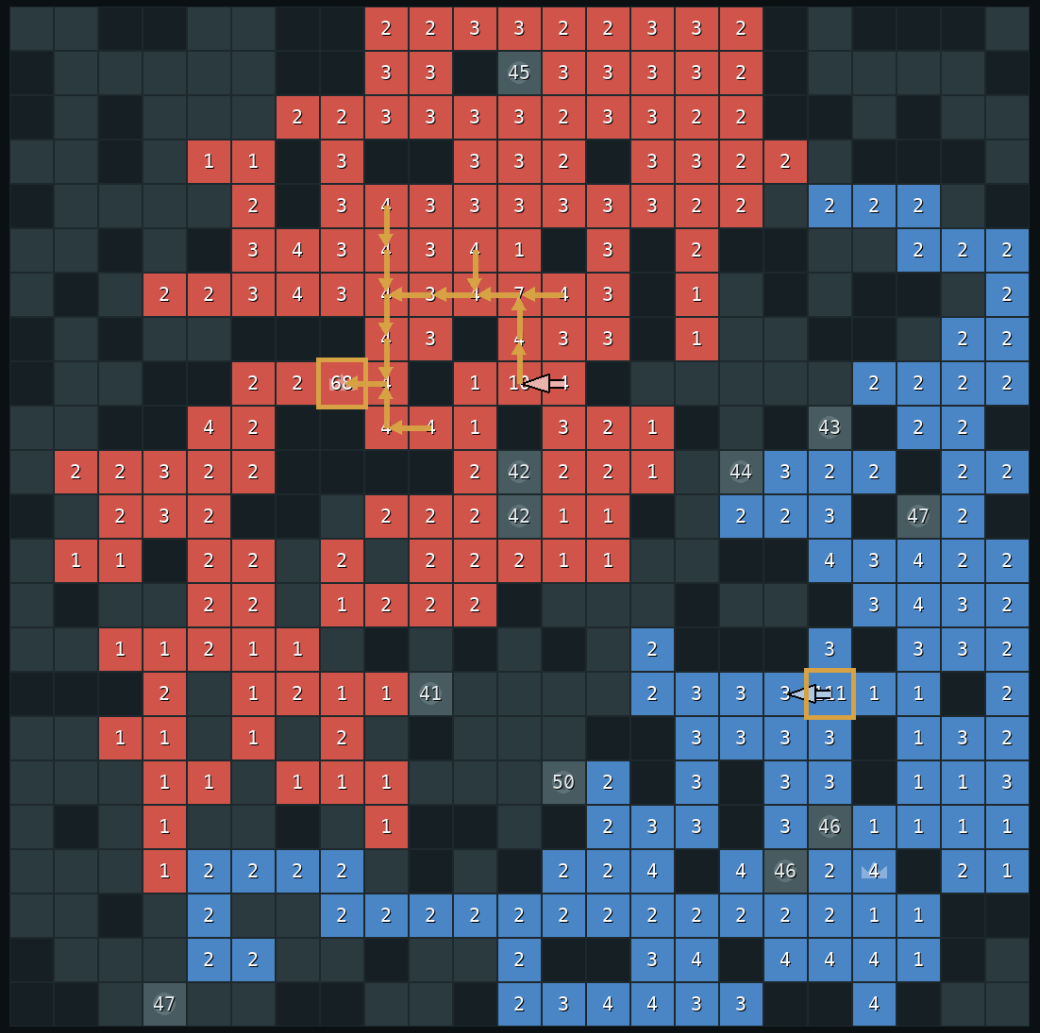
A big enemy stack moved 2+ squares closer in the last 5 turns. Its distance = turns left.

2 Measure

When it arrives, how many soldiers short will the general be?

3 Gather

Use every remaining move to pull soldiers into the general.



Right: the 14-move plan at turn 219 (gold arrows), all ending in the general.

IDEA 4

Which stacks to pull in, and in what order

In words: collect as many soldiers as possible into the general, using no more moves than the turns left.

$$\text{maximise} \quad \sum_{v \,\in\, \text{tree}} (a_v - 1) \;-\; \lambda \cdot \#\text{moves} \qquad \text{with} \quad \#\text{moves} \le \text{ETA} - 2$$

a_v − 1 soldiers we can take from square v (one must stay)

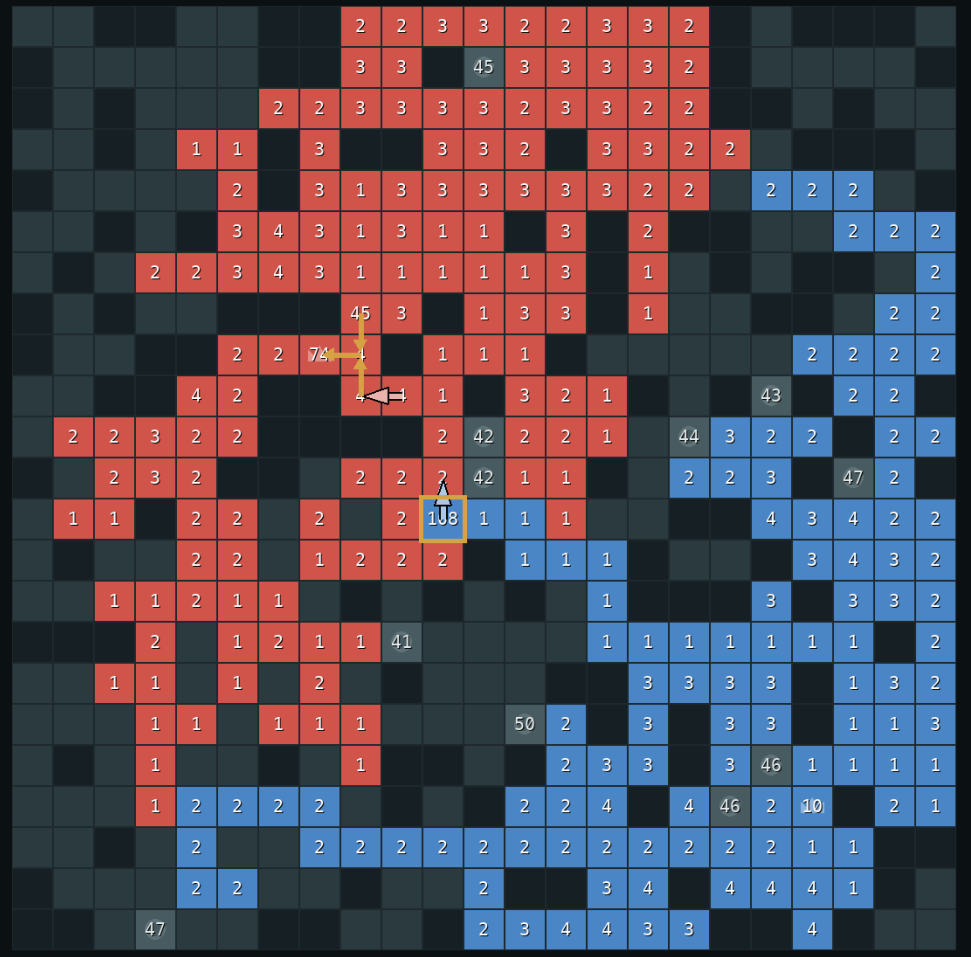
λ a price per move. Try prices until the plan fits the time left.

ETA − 2 turns before the attacker arrives, minus a small safety margin

Order: start with the farthest stacks, so soldiers pile up on the way in.

TERM Prize-collecting Steiner tree · binary search

A tree of routes into one point that collects the most prize for its cost (NP-hard). Binary search finds the right price per move by halving the range each try: 7 tries.



Turn 230: three moves of the plan left.

Choosing when you don't know the opponent

Too many single moves to search, so search over a few plans instead, each played forward in simulation (a rollout).

Then decide. Average over what we think the opponent is (expected value), or assume the worst (maximin), or mix the two (the Hurwicz criterion).

Expected value: average over opponents, weighted by our belief

$$\mathrm{EV}(a) = \sum_b P(b)\, V(a,b)$$

Maximin: assume the worst opponent

$$\mathrm{MM}(a) = \min_b V(a,b)$$

Hurwicz: a mix of the two; ours is half and half

$$H(a) = \alpha\,\mathrm{EV}(a) + (1-\alpha)\,\mathrm{MM}(a), \qquad \alpha = \tfrac{1}{2}$$

THE PAPER

Barriga, Stanescu & Buro, AIIDE 2015 (Puppet Search)

Searches over the choices inside a few scripted plans instead of over raw moves, which makes look-ahead affordable in real-time strategy games.

EXAMPLE turn 240 of the game

$$H(\text{castle}) = \tfrac{1}{2}(0.70) + \tfrac{1}{2}(0.69) = 0.69 \;>\; H(\text{expand}) = 0.59$$

IDEA 5

Pick a plan by imagining the next 40 turns

FROM THE BOOK looking ahead in games; choosing under uncertainty

FROM THE PAPER Barriga, Stanescu & Buro 2015 (Puppet Search)

plan	vs rusher	vs castle rusher	vs land grabber	vs waiter	score
keep expanding	0.83	0.83	0.59	0.83	0.59
take castle A	0.88	0.88	0.69	0.88	**0.69**
take castle B	0.83	0.83	0.69	0.83	0.69
take castle C	0.87	0.87	0.69	0.87	**0.69**
attack now	0.83	0.83	0.59	0.83	0.59

Real numbers from turn 240: the learned win estimate (section 3) after 40 imagined turns.

In words: half the average (weighted by our guess about the opponent), half the worst case.

$$\text{score}(a) = \tfrac{1}{2}\sum_b P(b)\,V(a,b) \;+\; \tfrac{1}{2}\min_b V(a,b)$$

Good against the opponent we expect, never a disaster if we guessed wrong.

TERM **Rollouts · Hurwicz criterion**

A rollout plays the game forward in simulation. The Hurwicz criterion scores a choice by mixing its average and its worst case; ours is 50/50.

IDEA 6 · THE CONCEPT

Is an investment worth it?

A castle is an investment: you pay soldiers now, and it produces soldiers until the game ends.

Compare what it will produce with what it costs. And to know whether you need it at all, project both players' armies forward at their current income.

Income per turn: generals and castles give ½, each square of land 1/50

$$\iota = \tfrac{1}{2}\left(\#\text{generals} + \#\text{castles}\right) + \tfrac{\text{land}}{50}$$

Projected army after L more turns

$$A(L) = A_0 + \iota \cdot L$$

A castle pays if what it produces beats its cost (garrison g plus walk d)

$$\tfrac{1}{2}\left(L - d\right) > g + d$$

THE PAPER

Churchill & Buro, AIIDE 2011

Planned StarCraft build orders by simulating each player's economy forward in time, to see which investments pay off and when.

EXAMPLE 200 turns left, garrison 45, walk 10

$$\tfrac{1}{2}(200 - 10) = 95 \;>\; 45 + 10 = 55$$

IDEA 6

Castles: only when they pay and home stays safe

FROM THE BOOK weighing costs against expected payoff

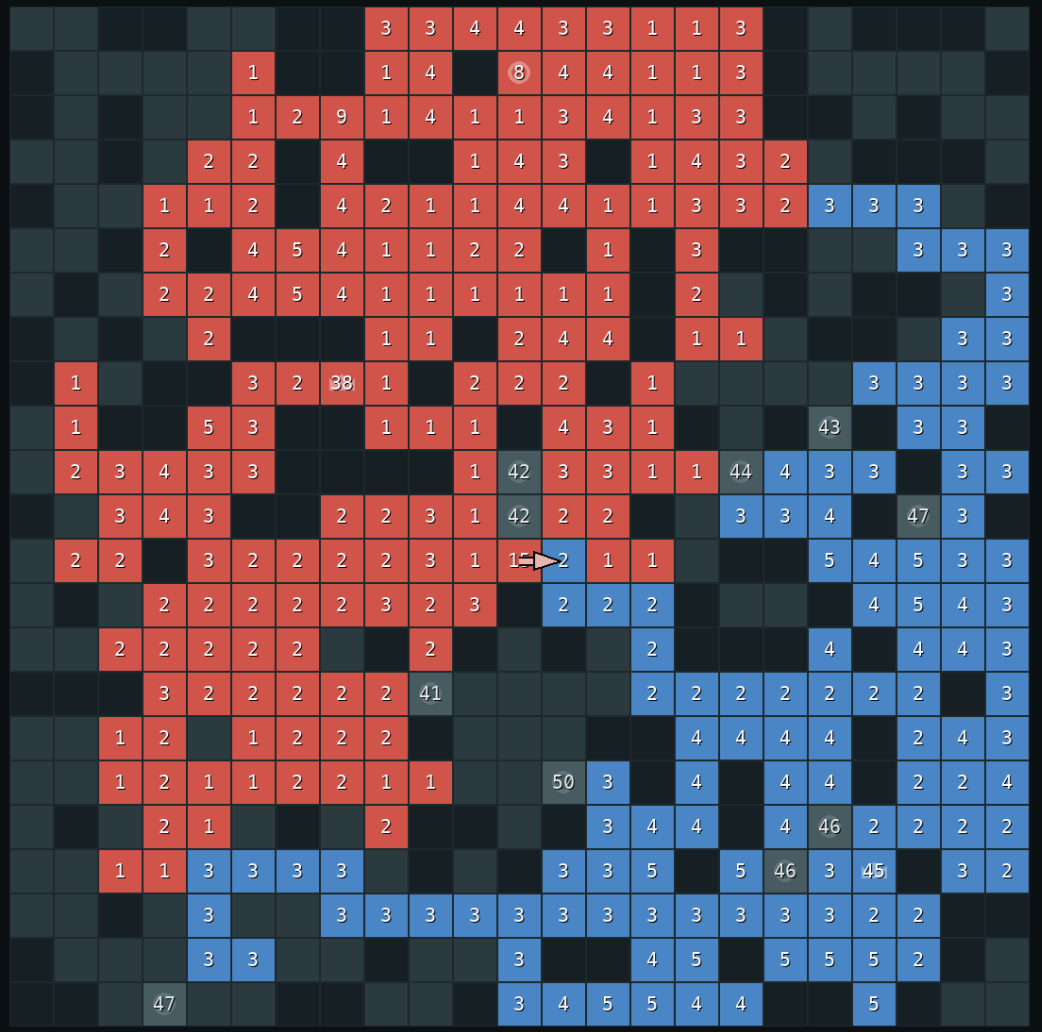
FROM THE PAPER Churchill & Buro 2011

Worth it? A castle produces like 25 squares of land. It must pay back its cost before the game ends.

Safe? Fast-forward to when our stack has been spent on the castle. Is the general still safe then?

Needed? If we are already clearly ahead, take only cheap castles.

Right: turn 298, the bot's first castle in this game.



TERM **Expected utility (AIMA ch. 16)**

The value of a choice is what it is expected to gain minus what it costs, over the time it has to pay off.

IDEA 7 · THE CONCEPT

Exploiting a guess without betting the game on it

If you know your opponent's habits, you can play the best response to them. But if the guess is wrong, a best response can lose badly.

Safe play guarantees the game's value whatever the opponent does. Safe exploitation sits between the two: exploit only as much as you can afford to lose.

THE PAPER

Ganzfried & Sandholm, ACM TEAC 2015

Showed when it is safe to deviate from safe play to exploit an opponent: only up to what you have already gained above the game's value.

The value of the game: the best guarantee against any opponent

$$v^* = \max_{\sigma} \min_{\tau} u(\sigma, \tau)$$

Best response: the best play against a guessed opponent q

$$\mathrm{BR}(q) = \arg\max_{\sigma} u(\sigma, q)$$

Safe exploitation: never risk falling more than k below the value

$$\min_{\tau} u(\sigma, \tau) \geq v^* - k$$

EXAMPLE rock-paper-scissors, opponent plays rock 50%: paper earns

$$u(\text{paper}, q) = 0.5(+1) + 0.25(0) + 0.25(-1) = +0.25$$

IDEA 7

Use the guess, but never trust it with the general

FROM THE BOOK game theory: best response vs playing safe

FROM THE PAPER Ganzfried & Sandholm 2015

Opponent looks aggressive	keep 60% of each threat at home; counter-attacks allowed
Opponent looks quiet	keep 40% at home; grab more land
We are not clearly ahead	take castles, enemy castles first
We are clearly ahead	only cheap castles

The guess changes what the bot prefers.

It never changes the safety test.

A wrong guess costs some efficiency, never the general.

TERM **Best response vs maximin (AIMA ch. 18)**

Best response: best play against the opponent you expect. Maximin: best play against the worst one. We play best response with a maximin safety floor.

IDEA 8 · THE CONCEPT

The arithmetic of an attack

An attacking stack shrinks on the way: one soldier stays on every square it crosses, and it fights anything in its path.

The defending general keeps growing, and the enemy can bring other stacks home. The attack works only if what arrives beats all of that.

What arrives: the stack, minus one per step, minus what it fights

$$x_{\text{arrive}} = x - d - \sum_{\text{path}} \text{garrison}$$

The attack works only if it beats the general, its growth, and reinforcements

$$x_{\text{arrive}} > G + \lfloor d/2 \rfloor + R_{\text{them}}(d)$$

SOURCE

The game's own rules

No paper needed here: these are the movement and production rules of the game, written as inequalities. The attack is then checked with the AND-OR test of idea 3.

EXAMPLE (illustrative) a 60-stack, 10 steps, 5 in the way; general 2, reinforcements 20

$$60 - 10 - 5 = 45 \;>\; 2 + 5 + 20 = 27$$

IDEA 8

Attack only when it's sure to work

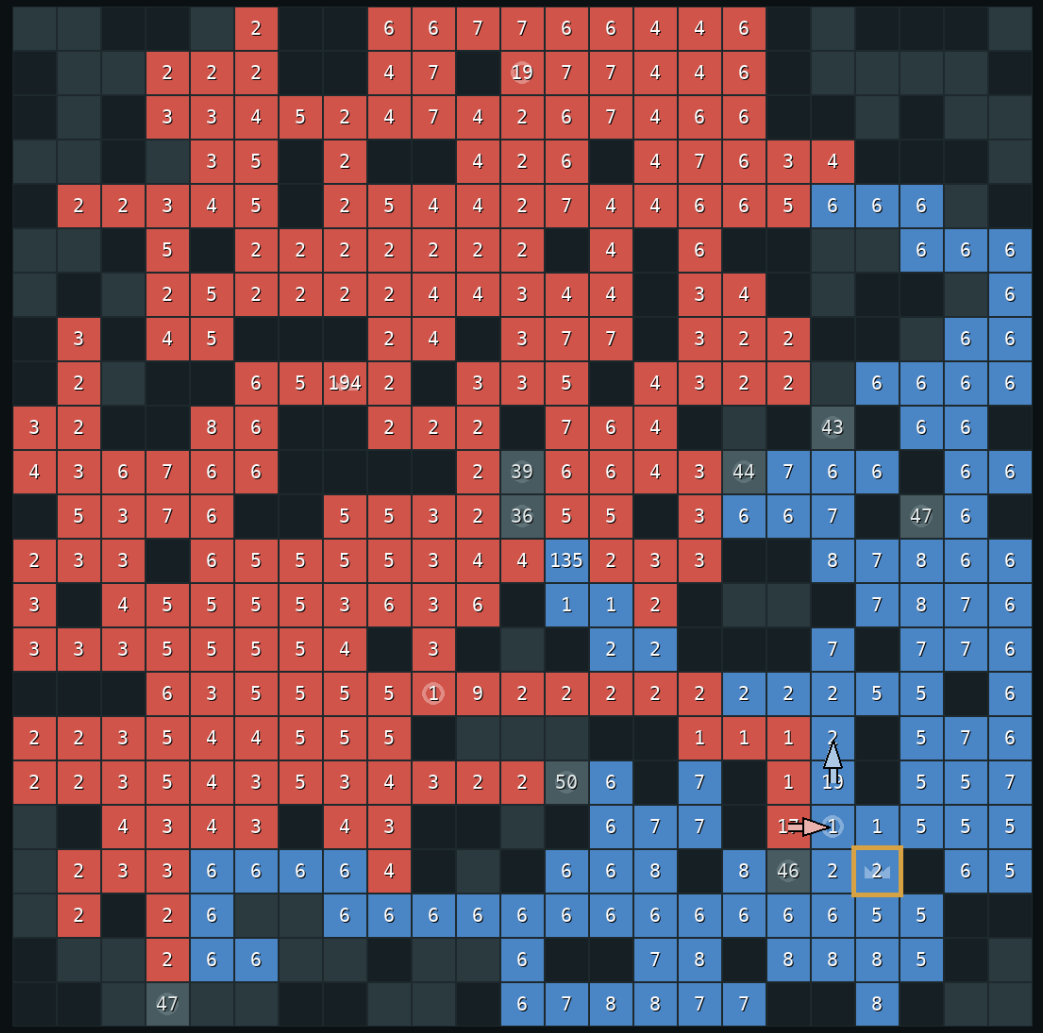
FROM THE BOOK planning against every possible reply

The bot attacks the enemy general only if:

1. its stack arrives bigger than everything the enemy can have there;
2. our own general stays safe while the stack is away;
3. it wins against four imagined replies.

In practice: when the enemy has emptied its general. Right: turn 405, the enemy general (gold box) has 2 soldiers.

Weakness: against a careful player it attacks late.



Where each idea came from

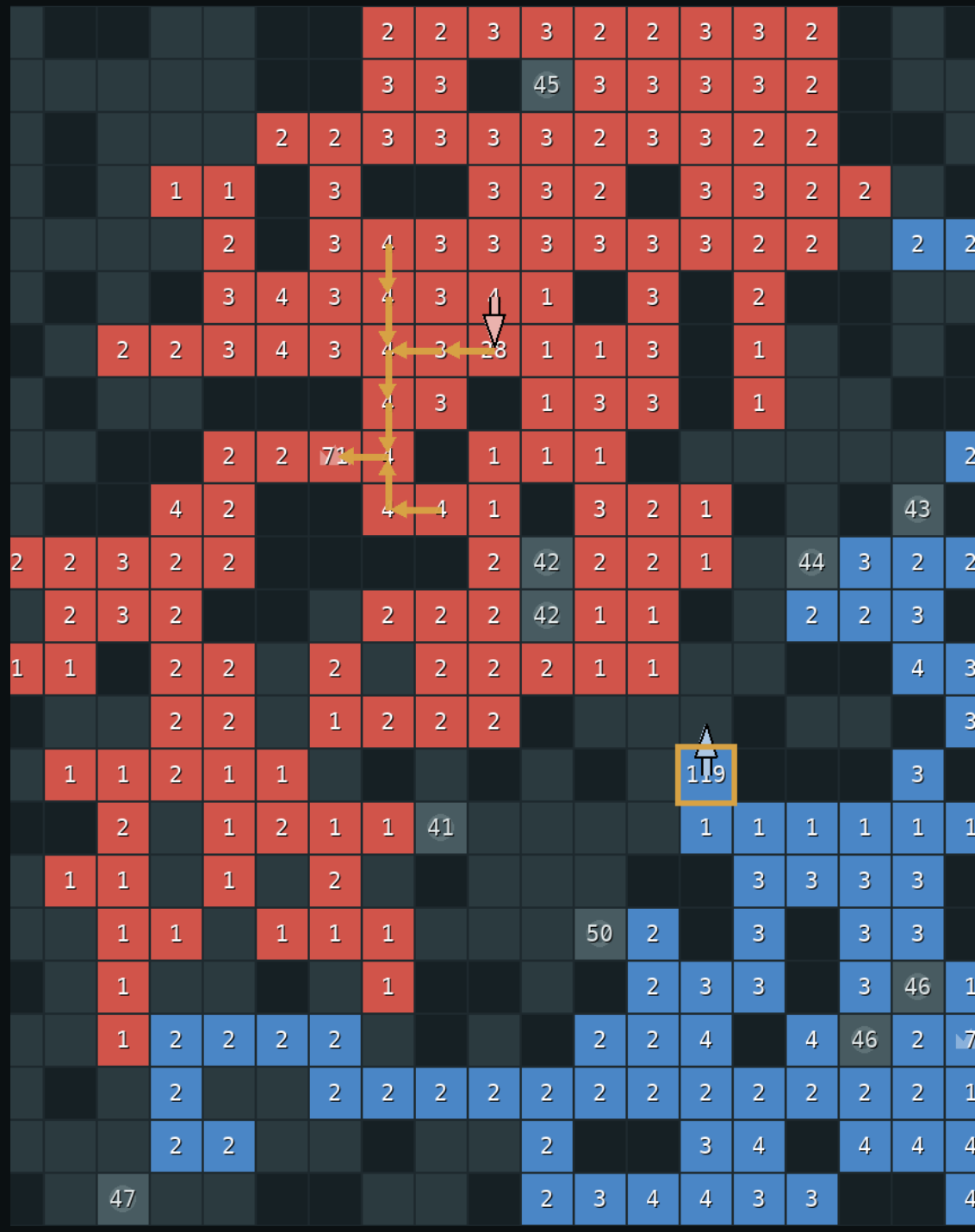
Paper	What we borrowed	Used in
Straka, Lisý & Schmid 2026	how to count soldiers that can be gathered in time	ideas 2, 4
Barriga, Stanescu & Buro 2015	search over a few plans, not every move	idea 5
Synnaeve & Bessière 2011	keep a probability over the opponent's strategy	idea 1
Weber & Mateas 2009	learn to recognise a strategy from game features	idea 1
Ganzfried & Sandholm 2015	exploit a guess only as far as is safe	idea 7
Churchill & Buro 2011	plan the economy by projecting income forward	idea 6

From the book: probability and belief updates, planning against replies, game lookahead, decisions under uncertainty, a small neural network.

3

Every part of v51

Everything the eight ideas skipped: the order of checks, the simulator, the learned win estimate, the classifier in full, and the small safety rules.



The full turn, in order

1	Read the board; update the guess about the opponent	always
2	Can a stack take their general right now?	take it
3	A big stack is coming and the general will be short?	fast defence (idea 4)
4	Update the castle gate and the threat gate (idea 7)	set the reserve R
5	Is our home losing the race?	counter-strike, if it passes the test
6	Urgent: safety score under the pad, or a proven attack?	v36 rules: attack / defend / pull home
7	A castle pays back and stays safe?	castle move (idea 6)
8	Otherwise	the committed plan (idea 5)

Every plan and castle move is tested first (idea 3). If it fails, the next rule is tried; if nothing passes, a simple defender script moves.

The simulator: where all the imagining happens

An exact copy of the game's rules in plain Python, checked against the real game engine on thousands of random moves: identical boards.

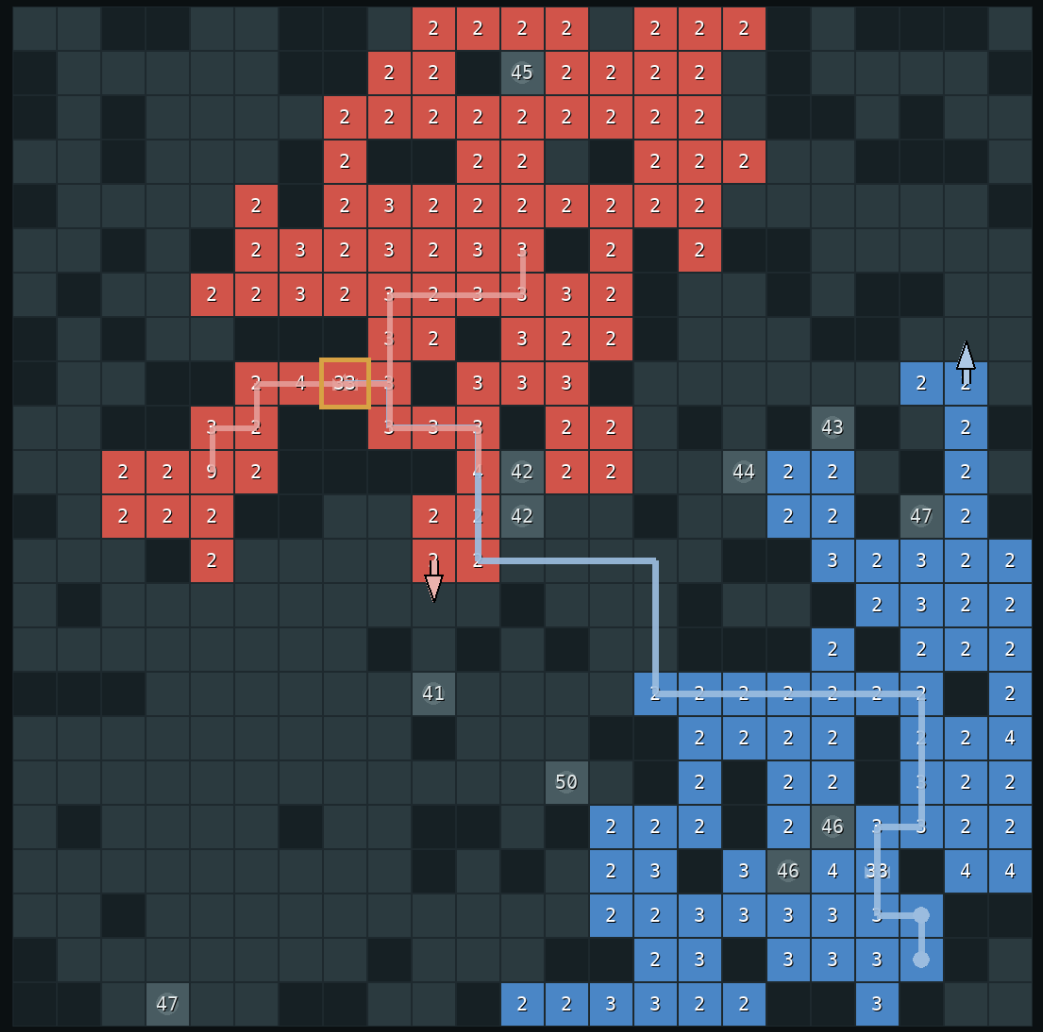
Rules: move one stack one square (all but one soldier, or half). Meeting stacks subtract. Take the general, win.

Cheap: one turn changes at most four squares, so only those are updated. Over a thousand simulated turns each time it picks a plan.

$$\text{general, castle}: +1 \text{ every } 2 \text{ turns}; \qquad \text{land}: +1 \text{ every } 50 \text{ turns}$$

TERM Transition model (AIMA ch. 3, 5)

RESULT(s, a): the state that follows state s after action a. Here a is a pair of moves, one per player, made at the same time.



Right: our stacks' routes home (light red) and enemy routes (light blue), turn 150.

Phases: how far ahead to look, how long to keep a plan

	opening	mid-game	end game
look ahead	24 turns	40 turns	60 turns
keep the plan	6 turns	10 turns	8 turns
castles offered	2 nearest	3 nearest	3 nearest
attack plans	no	yes	yes

End game: few turns left, or one side clearly ahead

$$\text{end} \Leftrightarrow L \leq 60 \ \vee \ (n \geq 2n' + 20 \ \wedge \ A \geq 2A' + 40)$$

Opening: no threat yet and plenty of free land (f free squares, n, n′ owned)

$$\text{opening} \Leftrightarrow \text{no threat} \ \wedge \ f > 0.35\,(f + n + n')$$

A new plan before K turns are up if:

$$\text{replan} \Leftrightarrow t \geq t_0 + K \ \vee \ \text{phase changed} \ \vee \ \text{castle changed hands} \ \vee \ G' < \tfrac{1}{2} G'_{\text{before}} \ \vee \ \text{plan done}$$

TERM **Depth-limited search (AIMA ch. 5)**

Look ahead a fixed number of moves, then score the position with an evaluation function instead of playing to the end.

Keeping a plan for several turns stops the bot from changing its mind every turn, which is how v11 lost.

The plan menu and the four imagined opponents

OUR PLANS

expand	grab free land with stacks that aren't on the general or a castle
hold	keep soldiers banked on the general and castles; expand with the rest
take castle ×2–3	one plan per nearby castle, nearest first (formula)
gather home	pull everything into the general; only when the safety score is low
strike	collect on our castle nearest their general, charge when the walk wins
rush	attack now with any stack whose walk already wins

Castles are offered in this order:

$$\text{rank}(c) = d(c) + \tfrac{1}{2}\,\text{garrison}(c)$$

IMAGINED OPPONENTS

rusher	walks its biggest stack at our general
castle rusher	takes castles, then rushes
land grabber	only expands
waiter	does nothing

8 plans × 4 opponents = 32 imagined games of 40 turns.

Plans that give identical boards against all four are merged: on average 6.1 plans become 4.6.

TERM **Macro actions (AIMA ch. 11)**

High-level actions that expand into many single moves, as in hierarchical planning.

Turning a board into a chance of winning

At the end of each imagined game the bot needs one number: how likely am I to win from here?

Logistic regression: add up a few measurements, each times a weight, then squash the total into 0 to 1 with the sigmoid. Each weight says how much one unit of a measurement changes the odds.

The win estimate: a weighted sum, squashed by the sigmoid σ

$$V(s) = \sigma(w^\top \phi(s)) = \frac{1}{1 + e^{-w^\top \phi(s)}}$$

Same thing read as odds: each weight adds to the log-odds

$$\log \frac{V}{1 - V} = w_1\phi_1 + w_2\phi_2 + \cdots + w_9\phi_9$$

Trained by minimising log-loss (with a small penalty on big weights)

$$\mathcal{L}(w) = -\tfrac{1}{n}\sum_i [y_i \log p_i + (1 - y_i)\log(1 - p_i)] + \tfrac{\lambda}{2}\|w\|^2$$

FROM THE BOOK

Russell & Norvig, AIMA ch. 19 and ch. 5

Chapter 19: logistic regression, a linear model with a sigmoid output, fitted by minimising log-loss. Chapter 5: an evaluation function scores positions where the search stops.

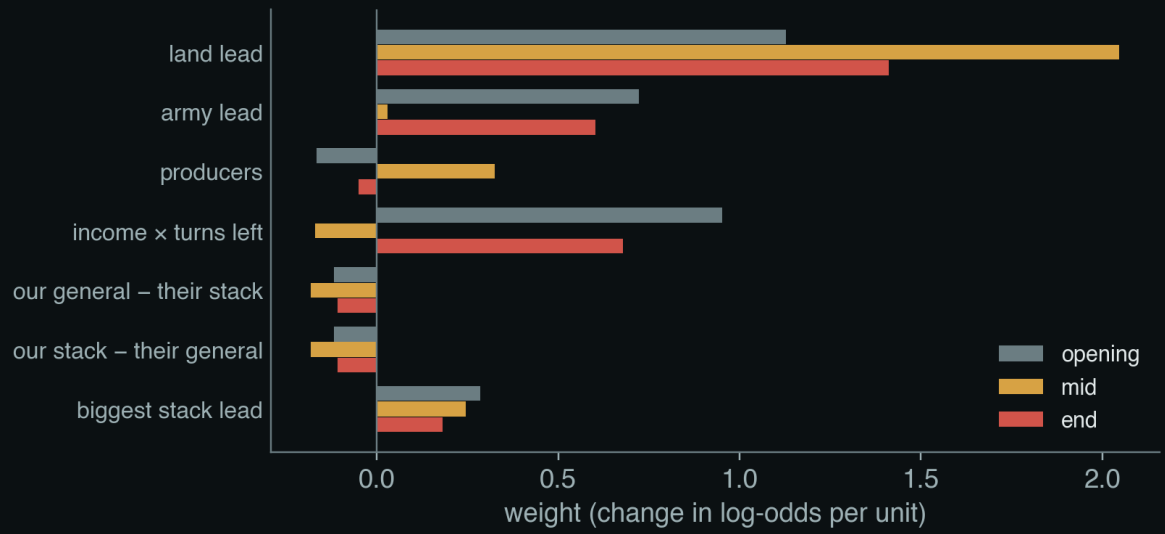
EXAMPLE (illustrative) opening: land lead +20, army lead +50, income lead worth +0.5

$$z = 1.13(0.4) + 0.72(0.5) + 0.95(0.5) = 1.29, \qquad V = \sigma(1.29) = 0.78$$

The nine measurements and what the model learned

`land`	land lead	÷ 50
`army`	army lead	÷ 100
`prod`	producers (generals + castles) lead	×1
`income`	income lead × turns left	÷ 100
`defence`	our general − their biggest stack	÷ 50
`attack`	our biggest stack − their general	÷ 50
`big`	biggest stack lead	÷ 50
`left`	turns left	÷ 500
`bias`	constant	1

Income = ½ per general or castle + 1 per 50 squares of land, per turn.



Opening: income × turns left counts most; early income compounds.

Mid-game: land dominates (weight 2.05).

Constant and turns left: exactly 0. Every position is seen from both sides, so neither side can be favoured.

Training it, and how well it predicts

Data: 7 scripted bots play each other on 200 maps; games of 300, 500 or 800 turns.

Every 5 turns: both players' measurements, labelled with who won. Draws dropped.

Split by map: 160 maps train (27,208 positions), 40 maps held out (6,986).

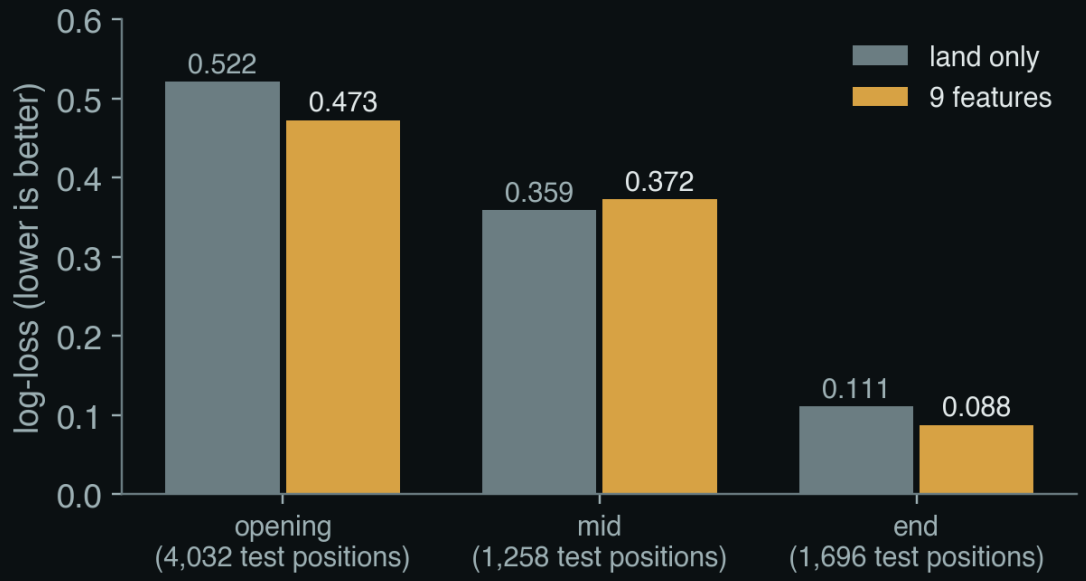
Fitted by Newton's method, one model per phase:

$$w \leftarrow w - H^{-1}\nabla\mathcal{L}$$

$$\nabla\mathcal{L} = \tfrac{1}{n}X^{\top}(p-y) + \lambda w, \qquad H = \tfrac{1}{n}X^{\top}SX + \lambda I, \qquad S = \operatorname{diag} p_i(1-p_i)$$

TERM **Newton's method (AIMA ch. 4.2)**

Uses the curvature H of the loss to jump towards the minimum. Log-loss is convex, so it reaches the one best answer in a few steps. Plain numpy: 9 weights.



Opening: clearly better than land alone; right 75% of the time (land alone: 71%).

Mid-game: no better than land alone on new maps.

End game: right 98% of the time.

Where the opponent weights in idea 5 come from

1 · Did each imagined opponent predict the last move?

$$w_m \leftarrow 0.95\, w_m \frac{P(\text{move} \mid m)}{0.5} + 0.05, \qquad P = 0.8 \text{ if } m \text{ predicted it, else } 0.2$$

2 · Combined with the classifier's ten styles:

$$w'_m \propto \sqrt{\frac{w_m}{\sum_k w_k}} \cdot \left(0.02 + \sum_{y \to m} P(y)\right)$$

A matching prediction counts four times as much as a miss; old evidence fades (0.95).

The square root gives one move less say than the classifier.

The 0.02 keeps every opponent possible, so the worst case never forgets one.

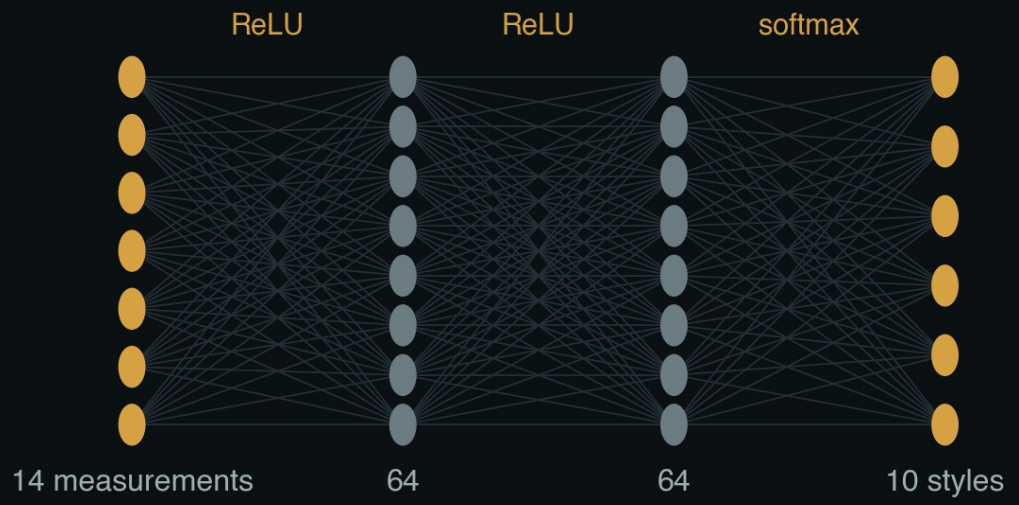
CLASSIFIER STYLE → IMAGINED OPPONENT

rusher; gathers early or mid-game	→ rusher
castle rusher	→ castle rusher
land grabber; gathers late; spreads out; economy bot; planner bot	→ land grabber
turtle	→ waiter

Fourteen measurements in, ten styles out

1	turn number	8	how much closer in 5 turns
2	land	9	land gained in 20 turns
3	total army	10	biggest stack's growth in 5 turns
4	share of army in its general	11	stacks of 5 or more
5	share in its biggest stack	12	times its general was emptied
6	castles held	13	army gained in 20 turns
7	biggest stack's distance to us	14	biggest stack within 10 squares?

All measured on the opponent, from the board alone, and scaled to roughly 0 to 1.



$$P(y \mid x) = \mathrm{softmax}(W_3\,\mathrm{ReLU}(W_2\,\mathrm{ReLU}(W_1\hat{x} + b_1) + b_2) + b_3), \quad \hat{x} = \frac{x - \mu}{\sigma}$$

$$(14 \cdot 64 + 64) + (64 \cdot 64 + 64) + (64 \cdot 10 + 10) \;=\; 5770 \text{ weights}$$

TERM **Feed-forward neural network (AIMA ch. 21)**

Layers of weighted sums with a simple non-linearity (ReLU) in between; the softmax turns the last layer into probabilities.

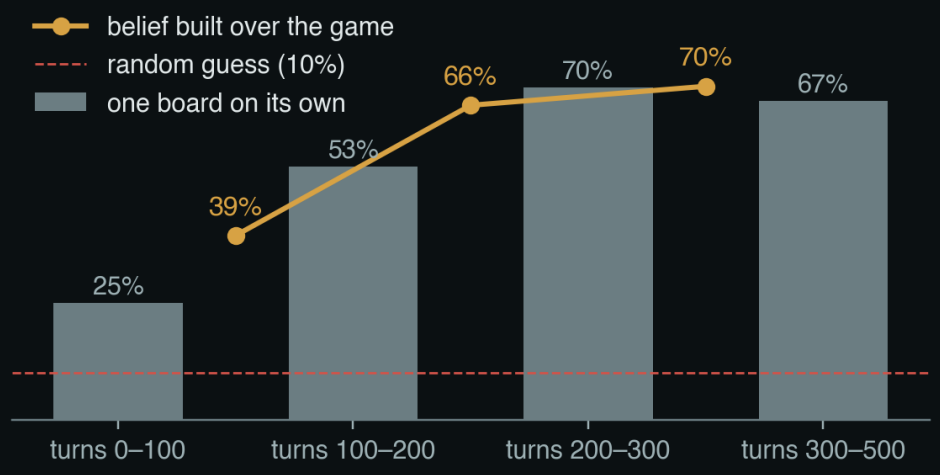
Training it, and where it gets confused

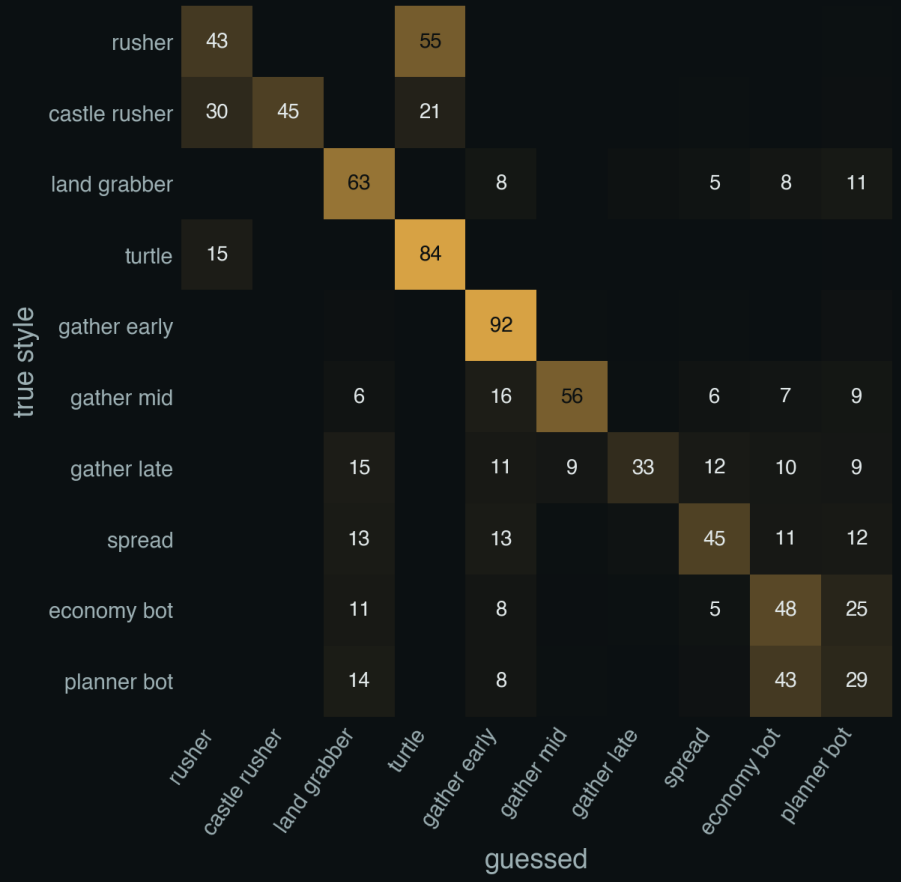
Data: v46 plays each of the 10 styles on 33 maps, both seats, 500 turns. Measurements every 2nd turn.

Split by map: 25 maps train (99,437 rows), 8 unseen maps test (33,318).

Training: Adam, learning rate 0.002, weight decay 0.0001, batches of 512, 60 passes, on the Mac GPU.

$$\mathcal{L}(\theta) = -\frac{1}{n}\sum_i \log P_\theta(y_i \mid x_i)$$





Rusher read as turtle (55%): both sit at home until the rusher launches. Economy bot vs planner bot: both map to the same imagined opponent.

The reserve: what the general must keep

From a lost game: a human walked their general 25 squares, collecting 84 → 137. Our safety score read +10 to +40 until two turns too late.

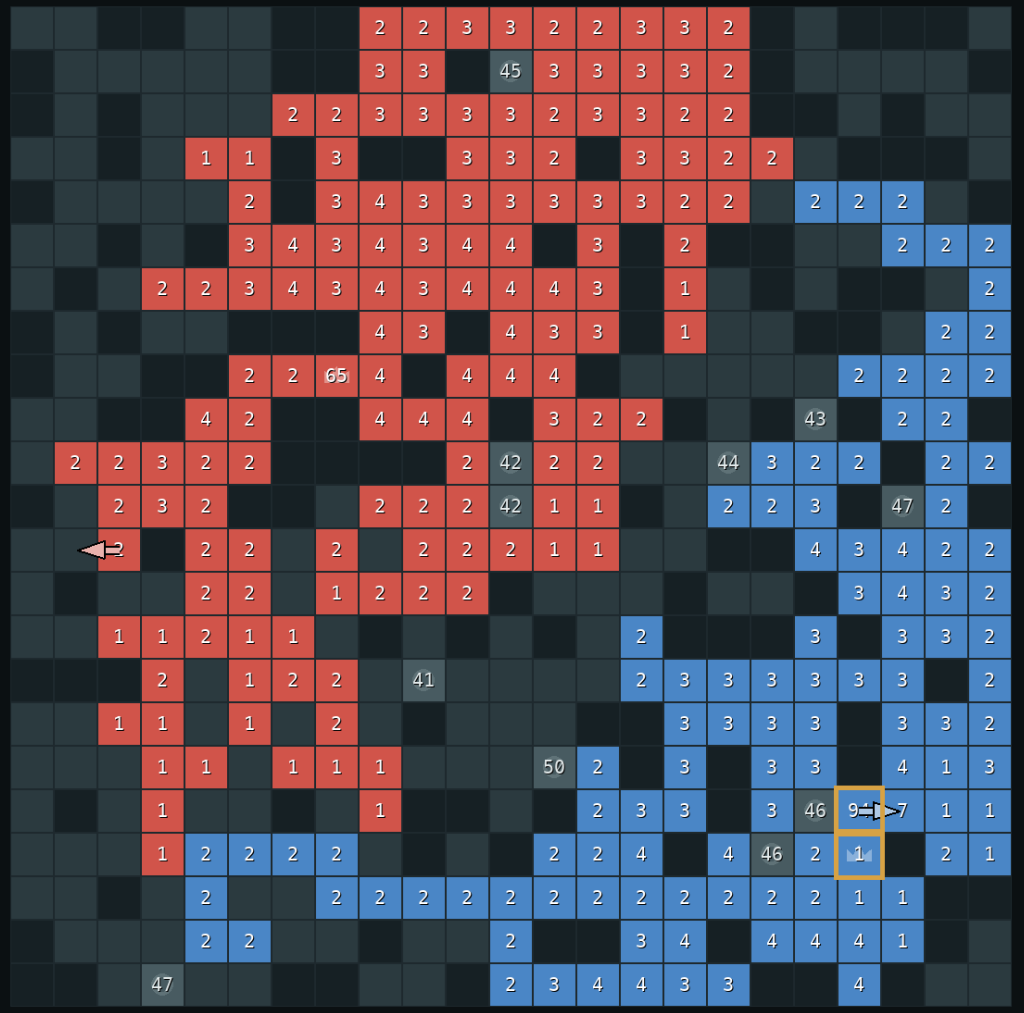
1 · Whatever leaves the general, it keeps a reserve, now and if the enemy waits:

$$G_{\text{keep}} + \text{growth}(H + d_s) \;\geq\; R \cdot x_s(H), \qquad H \in \{0, 30, 60\}, \;\; R \in \{0.4,\, 0.6\}$$

2 · The most the enemy could collect in H moves and walk at us:

$$M(H) \;=\; \sum_{H \text{ biggest}} (a_c - 1) \;+\; 1 + \text{growth}(H) \;-\; \text{cost}(\text{path})$$

R = 0.6 when the threat gate is open, **0.4** when it is shut (idea 7). Each collecting move picks up at most one square, so H moves collect at most the H biggest.



Turn 213 of our game: the enemy empties its general into one stack (gold boxes).

The pad, the lead, and counting each square once

The pad: how small is too small

$$\text{pad} \;=\; 3 + 0.08\,x_{\text{top}} + \max\!\left(0,\; -16\,\frac{\Delta m}{\Delta t}\right)$$

3 soldiers, plus 8% of their biggest moving stack, plus extra when the safety score has been falling (Δm/Δt over 8+ turns).

The lead: our stacks must land early

$$D_{\text{us}}(T - 2 - \lfloor T/10 \rfloor) \qquad \text{our stacks must land early}$$

In the safety score, our walks home get fewer moves than the enemy's: slack grows with the length of the walk.

Each square counted once

$$x_k \;\leftarrow\; x_k - \sum_{v \,\in\, P_k \cap C} (a_v - 1)$$

Walk k loses what earlier chosen walks (cells C) already picked up on its path P_k. Before this fix, a shared corridor was counted twice.

Two rules from replays of human wins

Never swap places with an attacker



Same turn: the general steps out, the attacker steps in. They pass each other, and the attacker lands on a general with 1.

So the general never steps toward an adjacent enemy stack. Another stack hits it, or the general waits and grows.

Counter-strike: when home will fall anyway

If the safety score says our general falls at turn T_fall, launch a stack at their general if it arrives first and wins:

$$d < T_{\text{fall}} = \min\{T : m_T \le 0\}$$

$$x_{\text{arrive}} > G' + \text{growth}(d) + D'_{\text{home}}(d-1) + 3$$

G′, D′_home: their general, and what they can pull home in time. +3 for safety.

Only with the threat gate open, and only if it passes the four-reply test (idea 3).

Castles in full

Pays back: a castle makes ½ a soldier per turn (g garrison, d walk, L turns left)

$$2\,(g+d)\;\le\;\min(L-d,\;400)$$

Which first: cheap and close; toward their general; enemy castles

$$\text{key} = g + d - 0.6\,\text{fwd} - 15\,[\text{enemy's}] + 2\,[\text{full send}]$$

Safe: fast-forward to the end of the walk

$$m\,(\text{after the walk},\;\;\text{enemy } d \text{ steps closer})\;>\;\text{pad} + 0.2\,x_{\text{top}}$$

Needed? The castle gate: project both armies to the end of the game

$$\text{lead} = (A - A') + (\iota - \iota')\,L; \quad \text{won} \Leftrightarrow \text{lead} > 0.25\max(50, A') \wedge \iota \ge \iota' \wedge n > n'$$

Candidates: castles within 30 squares, walk at most 25. Best 4 checked each turn.

Sources: the general (half, or all but one), an owned castle with 4+, or a big stack.

From the general: the reserve rule applies too.

Race won: only cheap castles (garrison + walk ≤ 30), never from the general, unless they hold castles.

Everyday play: expansion and the rule list

Expansion: score every square we could step into, take the best

$$\text{score} = 10\,\text{gain} + 0.1\,a_c + 0.5\,\text{free}(v) - 0.05\,d_{\text{enemy}}(v) - 5\,[\text{producer}] + 4\,[\text{threat route}]$$

gain: 1 for a free square, 2 for an enemy square we beat. The threat-route bonus blocks attack paths.

THE RULE LIST (v36): FIRST MOVE THAT PASSES THE SAFETY TEST WINS

1 proven attack	2 under attack: hit or pull home	3 short: pull home	4 castle walk	5 far ahead: concentrate
6 expand	7 walk to the edge	8 build up forward	9 general expands	10 gather home

At most 6 expensive tests per turn. If nothing passes: the defender script (hit an adjacent attacker we beat, else pull the best stack home, else wait).

Built, but switched off: learning across games

Across games, pick one of four settings and learn which works against this opponent (Exp3).

base	reserve +0, cheap castles ≤ 30
greedy	reserve −0.1, cheap castles ≤ 60
safe	reserve +0.1, cheap castles ≤ 20
castles	reserve +0, cheap castles ≤ 90

Off in the final bot: one game is noisy evidence, it would need many games against the same opponent, and test results would depend on game order. Every game today uses **base**.

Pick a setting: mostly by weight, 20% spread evenly

$$p_a = (1-\gamma)\frac{w_a}{\sum_b w_b} + \frac{\gamma}{K}, \qquad \gamma = 0.2,\ K = 4$$

After the game: reward r, scaled by how likely the pick was

$$w_a \leftarrow w_a \exp\left(\frac{\gamma r}{p_a K}\right)$$

r = 1 win, 0 loss, else 0.25 + 0.5 × our share of the land.

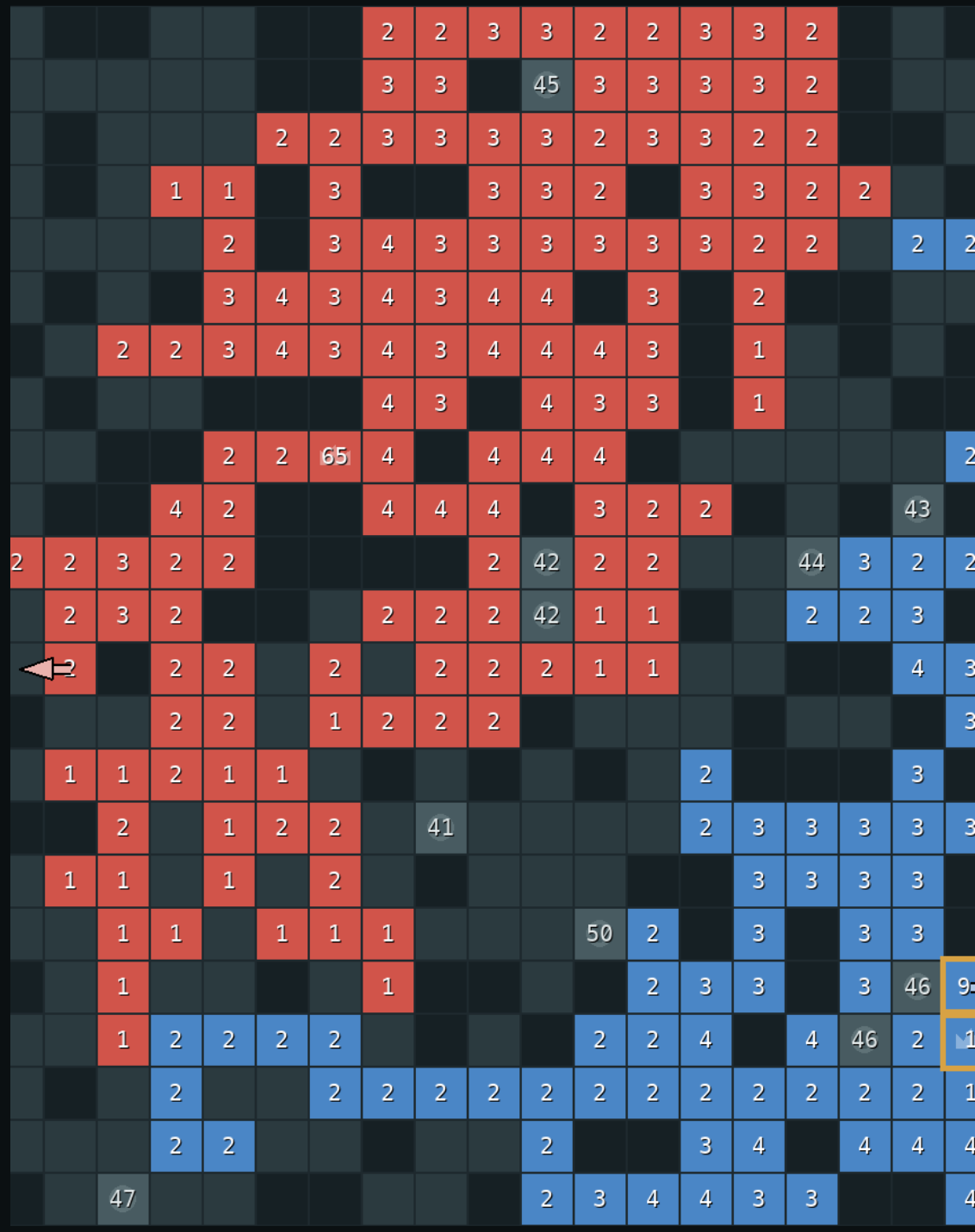
TERM **Multi-armed bandit (AIMA ch. 17)**

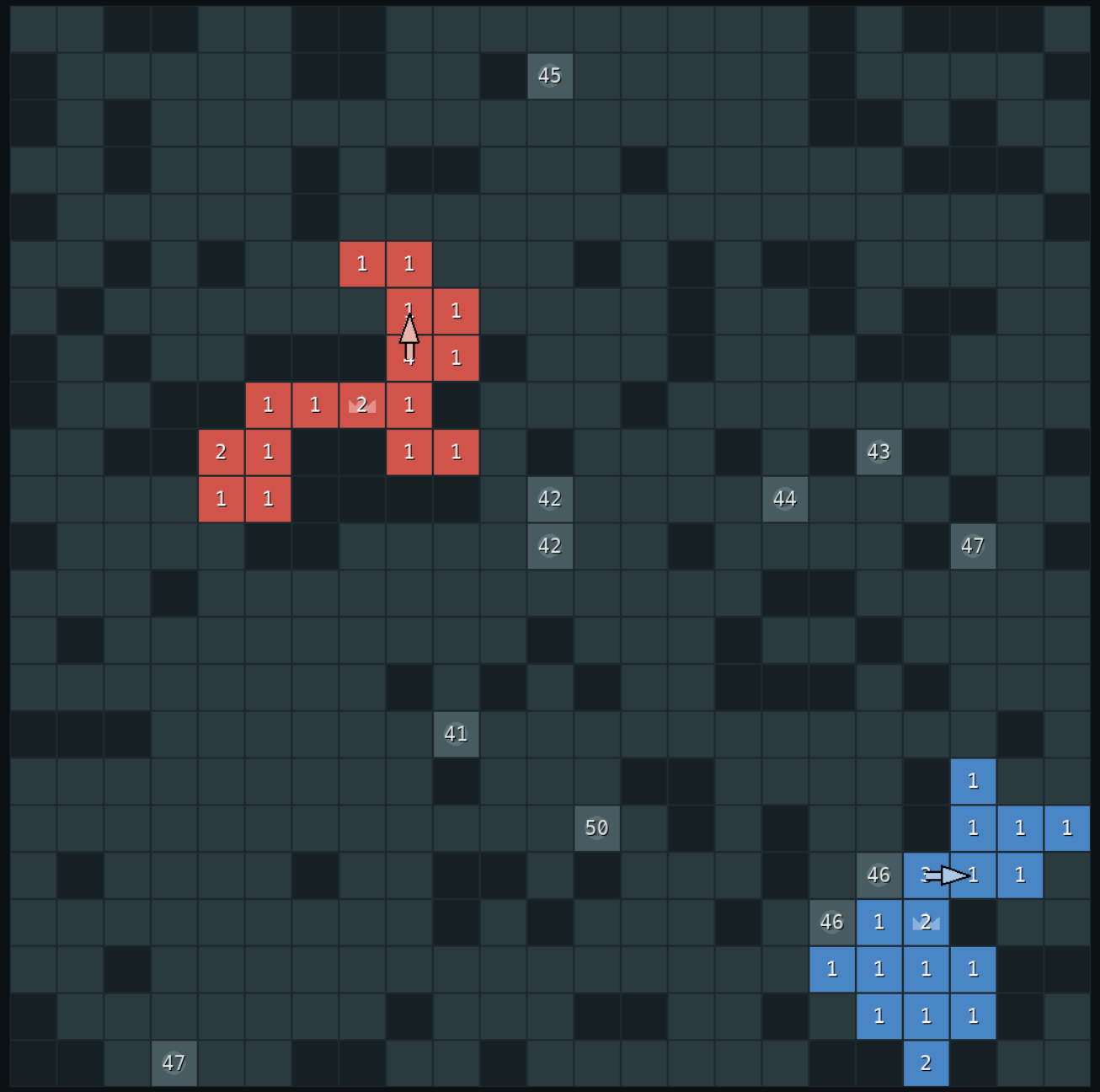
Repeatedly choose between options with unknown payoffs, balancing trying each (exploration) against using the best so far (exploitation).

4

One real game

Our bot (red) against a bot that plays the way people beat our earlier versions: grab land, pull everything into one stack, charge.





ONE REAL GAME

turn 40

Opening: grab land

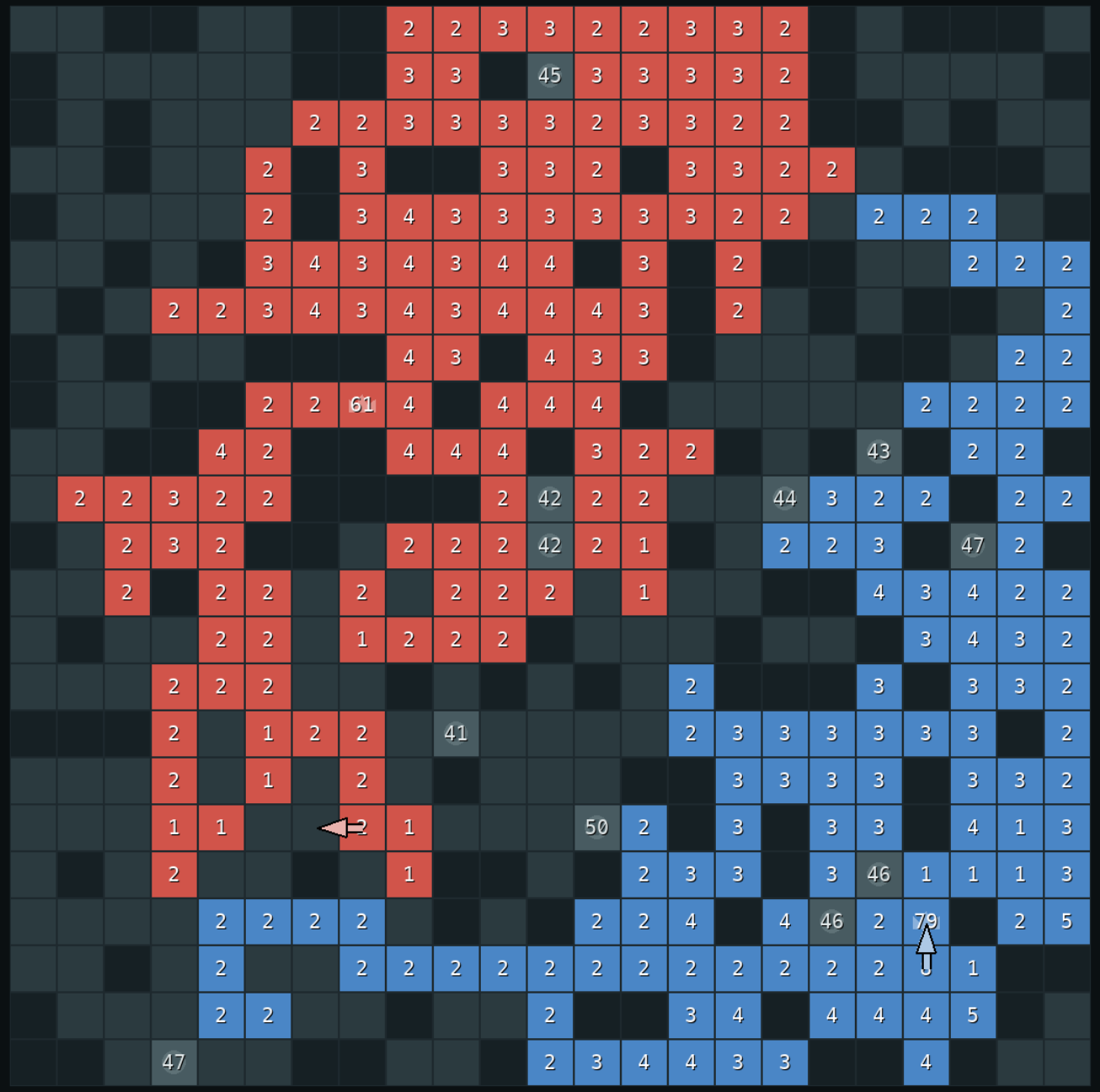
Both sides grab one square per move. Nobody is threatening anybody yet, so the bot just expands.

21

our soldiers

21

their soldiers



ONE REAL GAME

turn 205

The opponent starts collecting

Its general is filling up: it is pulling its soldiers in.

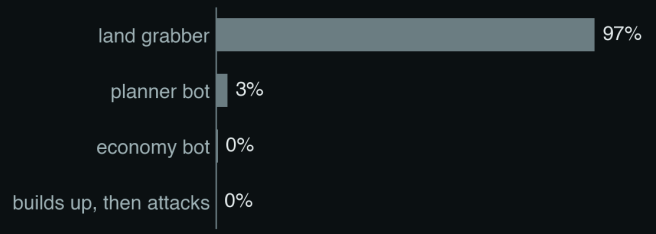
Our bot still thinks it's a land grabber, which so far is true.

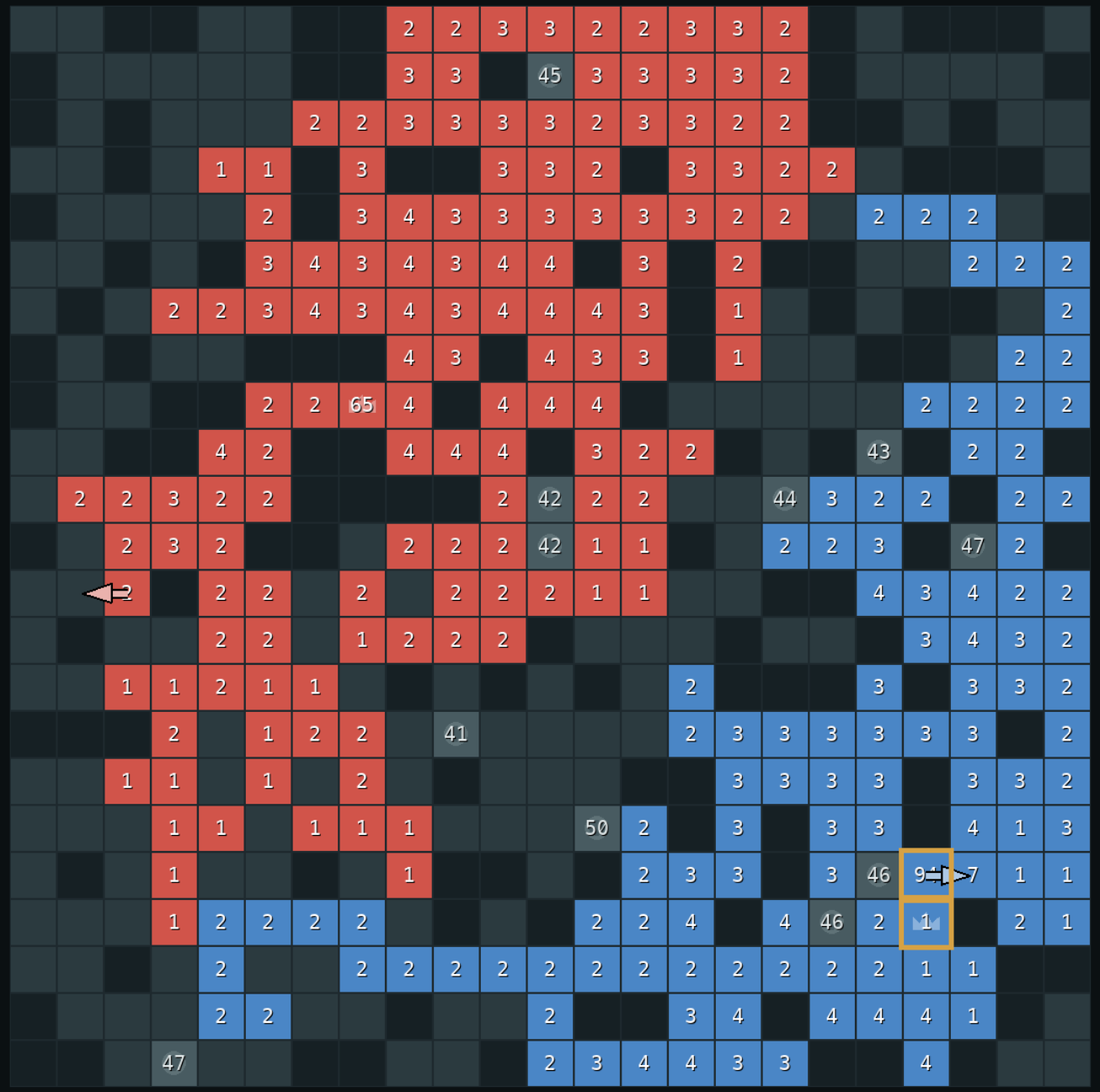
61

our general

79

their general





ONE REAL GAME

turn 213

It empties its general

Its general drops from 93 to 1. A 94-soldier stack appears (gold boxes).

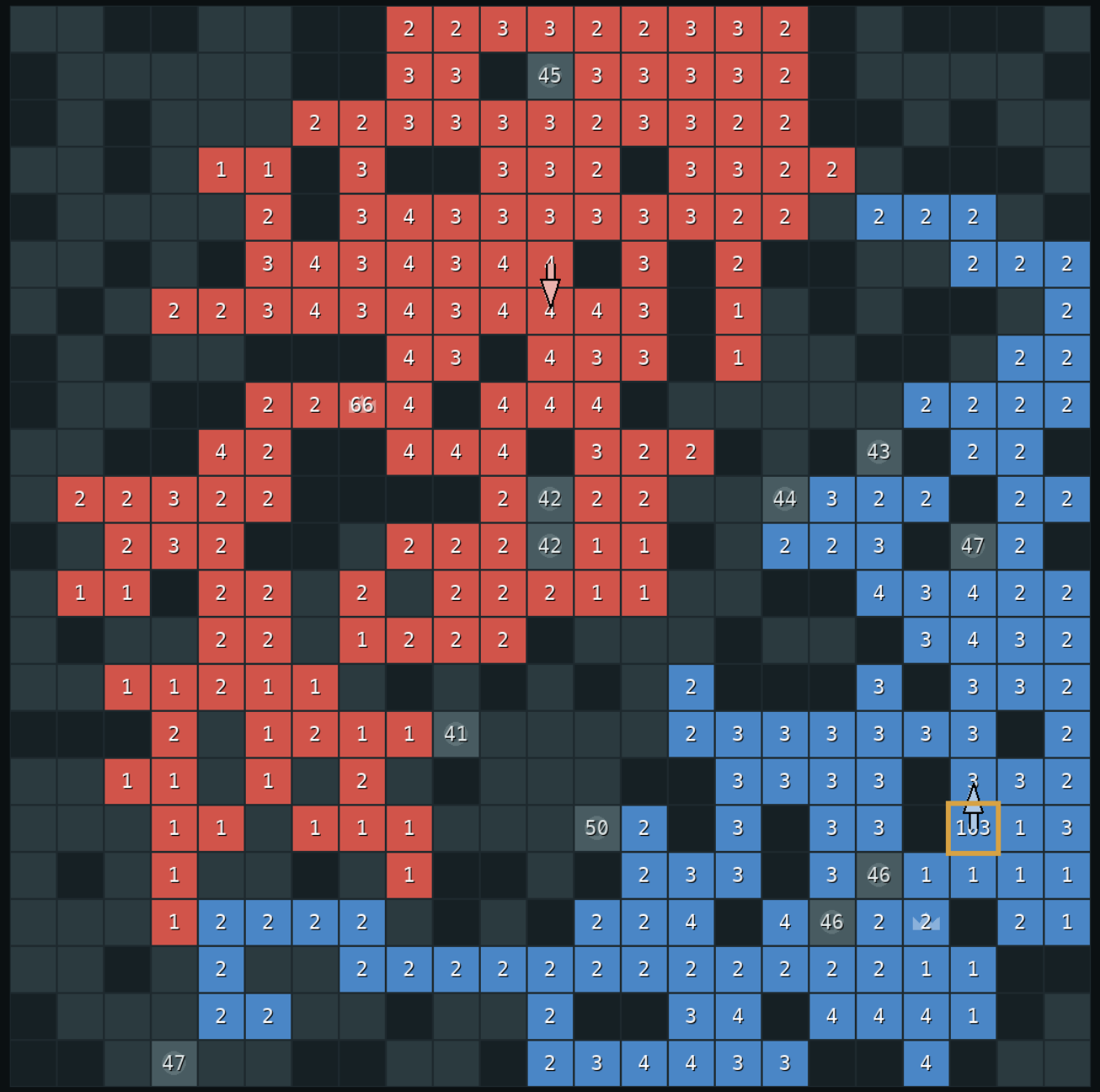
Our safety score drops to 12: an attack is possible soon.

94

their big stack

12

safety score



ONE REAL GAME

turn 215

The guess changes

Now the board looks like a player that builds up and attacks: 97%.

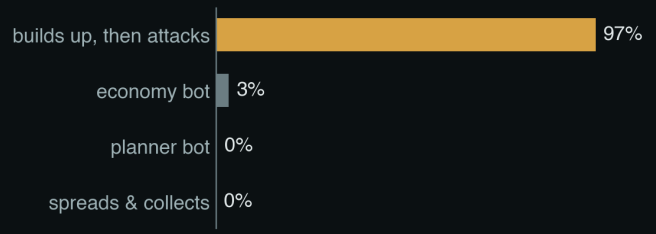
The bot makes its first defensive move.

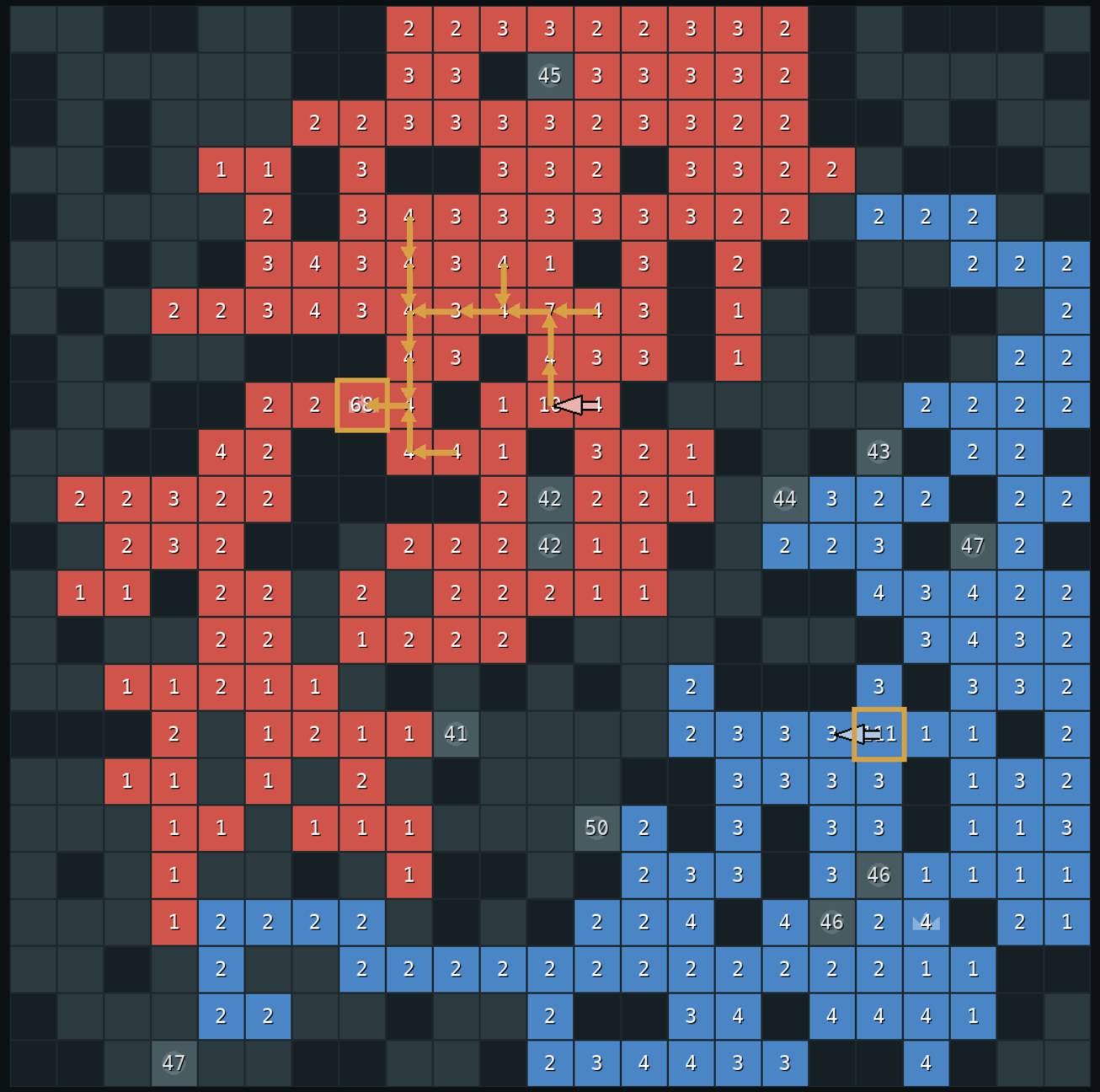
103

attacker

66

our general





ONE REAL GAME

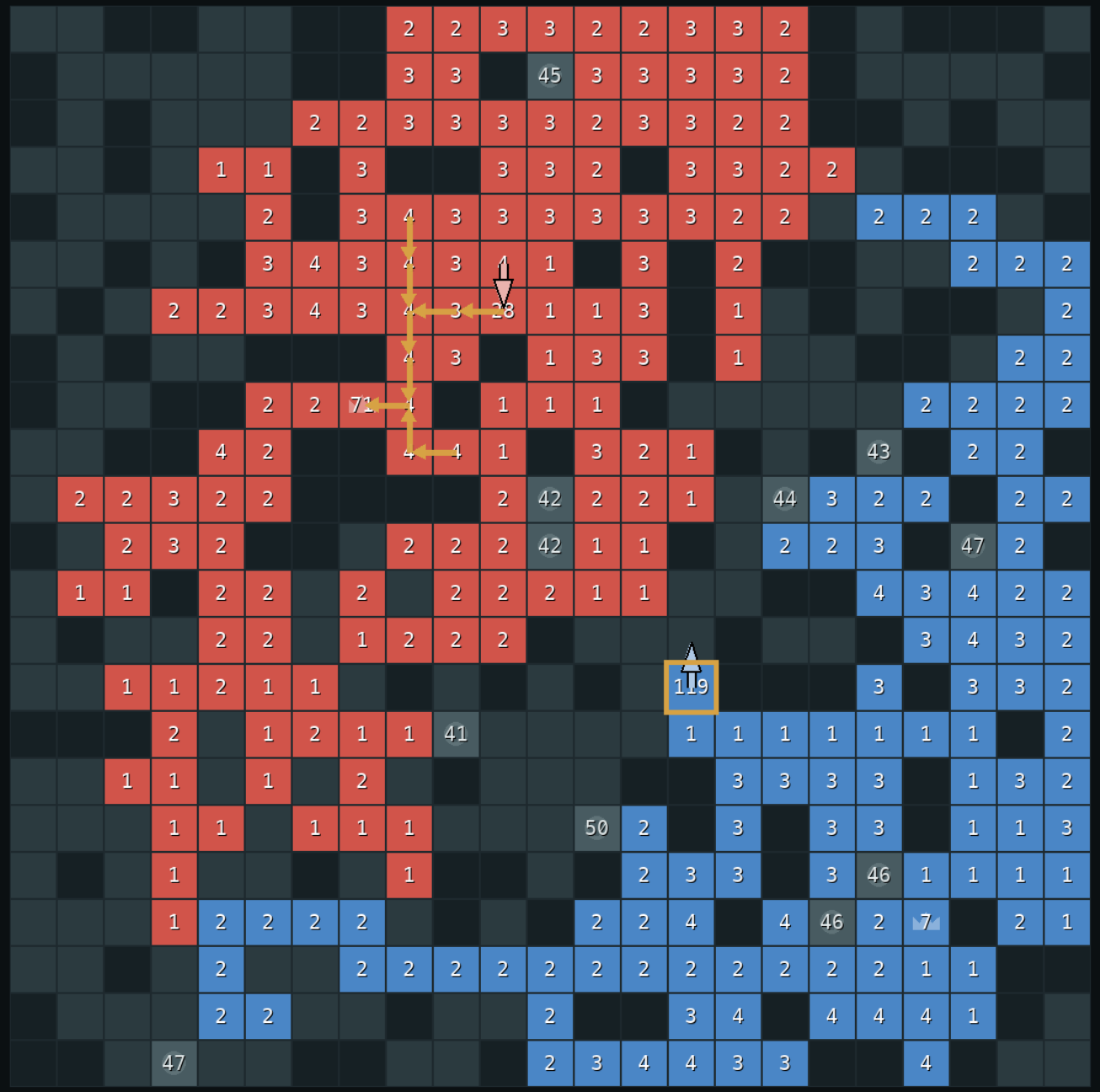
turn 219

Attack confirmed: plan the defence

The stack is 18 squares away and coming closer.

The bot plans 14 moves that pull soldiers into its general (gold arrows). Every move from now on is defence.

111	18	68
attacker	squares away	our general



ONE REAL GAME

turn 224

Pulling soldiers in

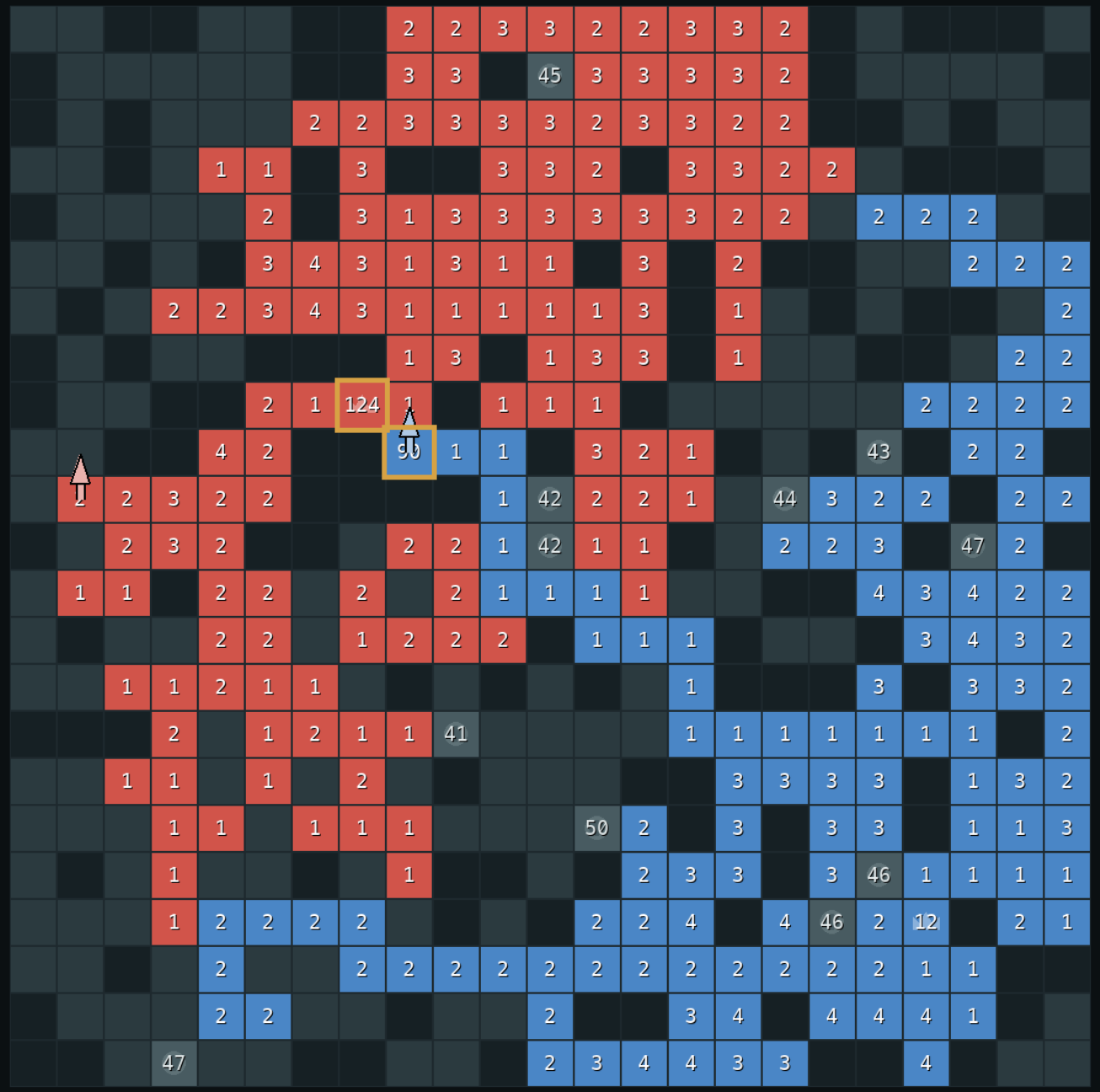
The farthest stacks move first, so soldiers pile up on the way in.

The attacker shrinks as it walks.

119 attacker

13 squares away

71 our general



ONE REAL GAME

turn 235

Ready

The big stack next to the general steps in. The general now has 124.

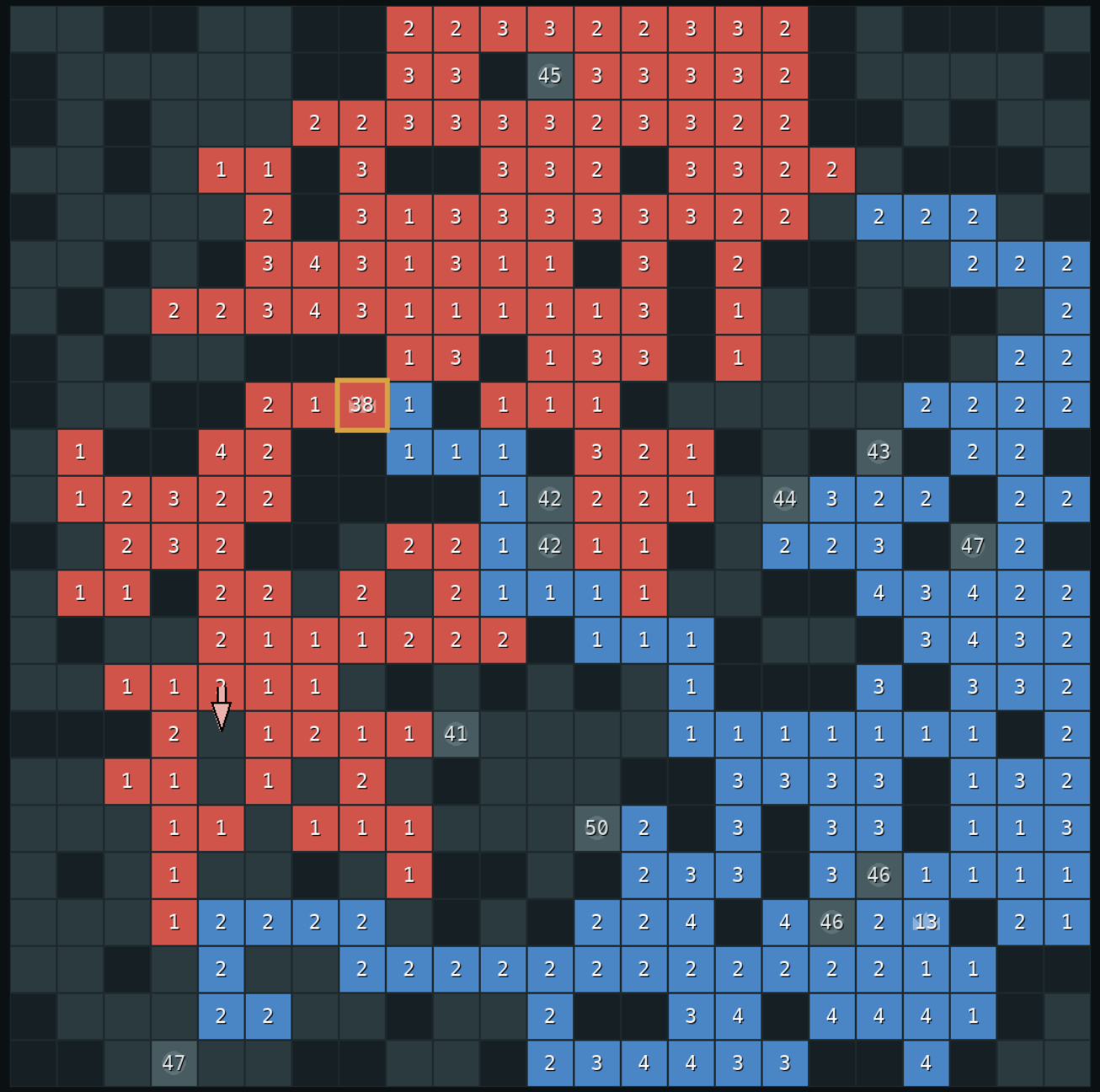
The attacker is right next to it with 90.

124

our general

90

attacker



ONE REAL GAME

turn 237

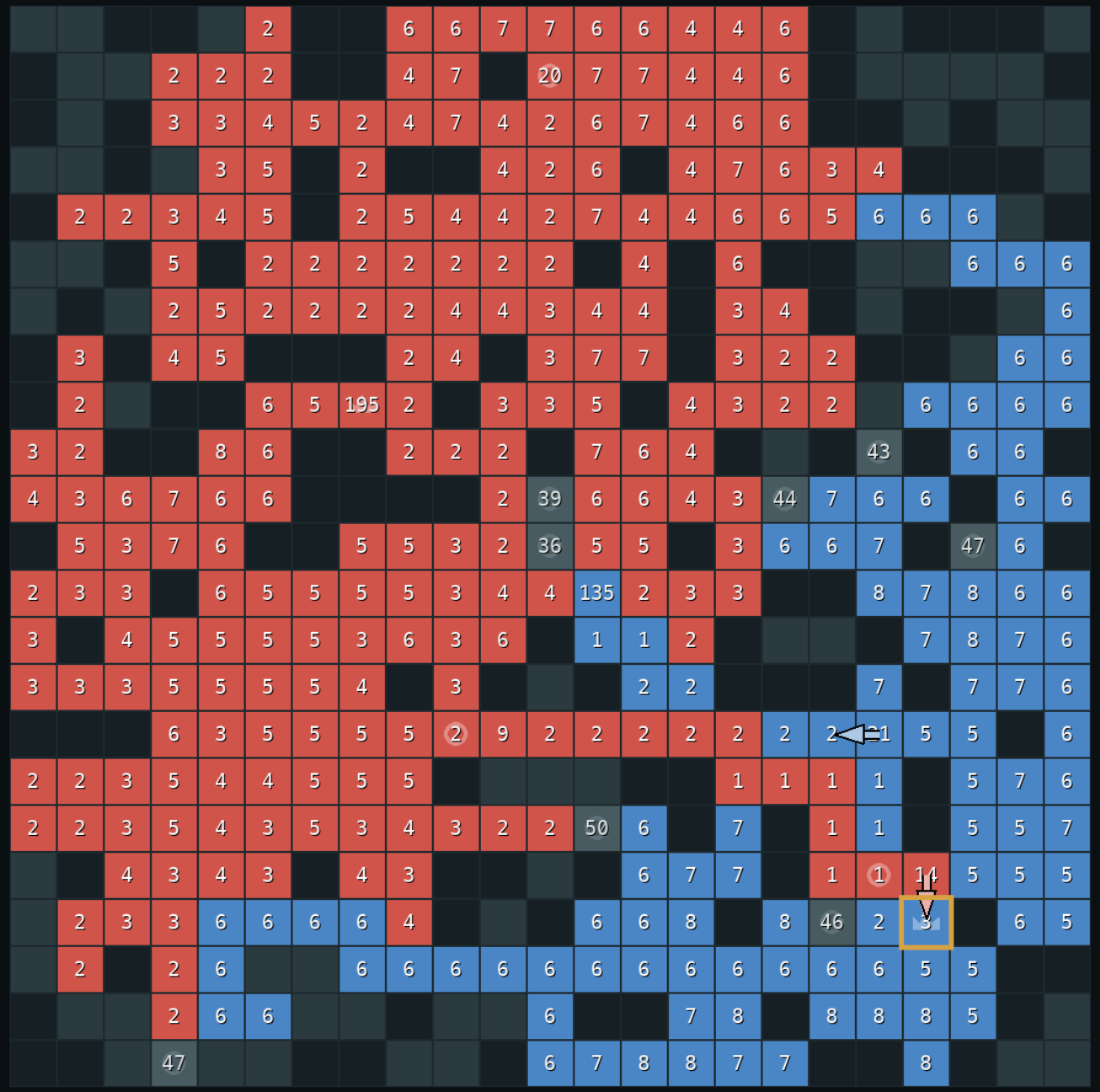
The attack fails

The attacker hits with 88. The general keeps 38.

Our previous bot lost this exact position: 77 against 82.

38
left in our general

0
attackers left



ONE REAL GAME

turn 407

Our attack, when it's sure

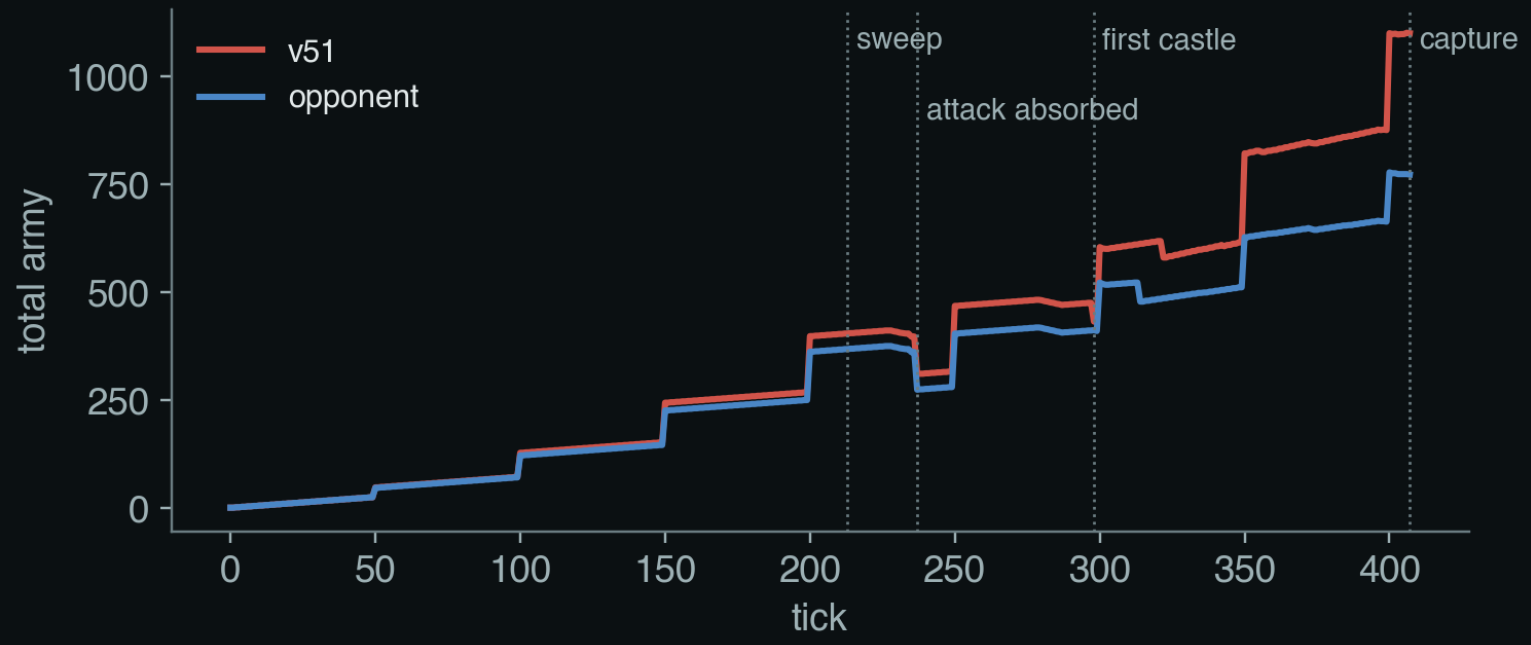
The bot went back to taking castles and pulled ahead.

At turn 405 the enemy general had 2 soldiers. The attack passed all four tests. Captured at 407.

1,099
our soldiers

772
their soldiers

The whole game in one picture



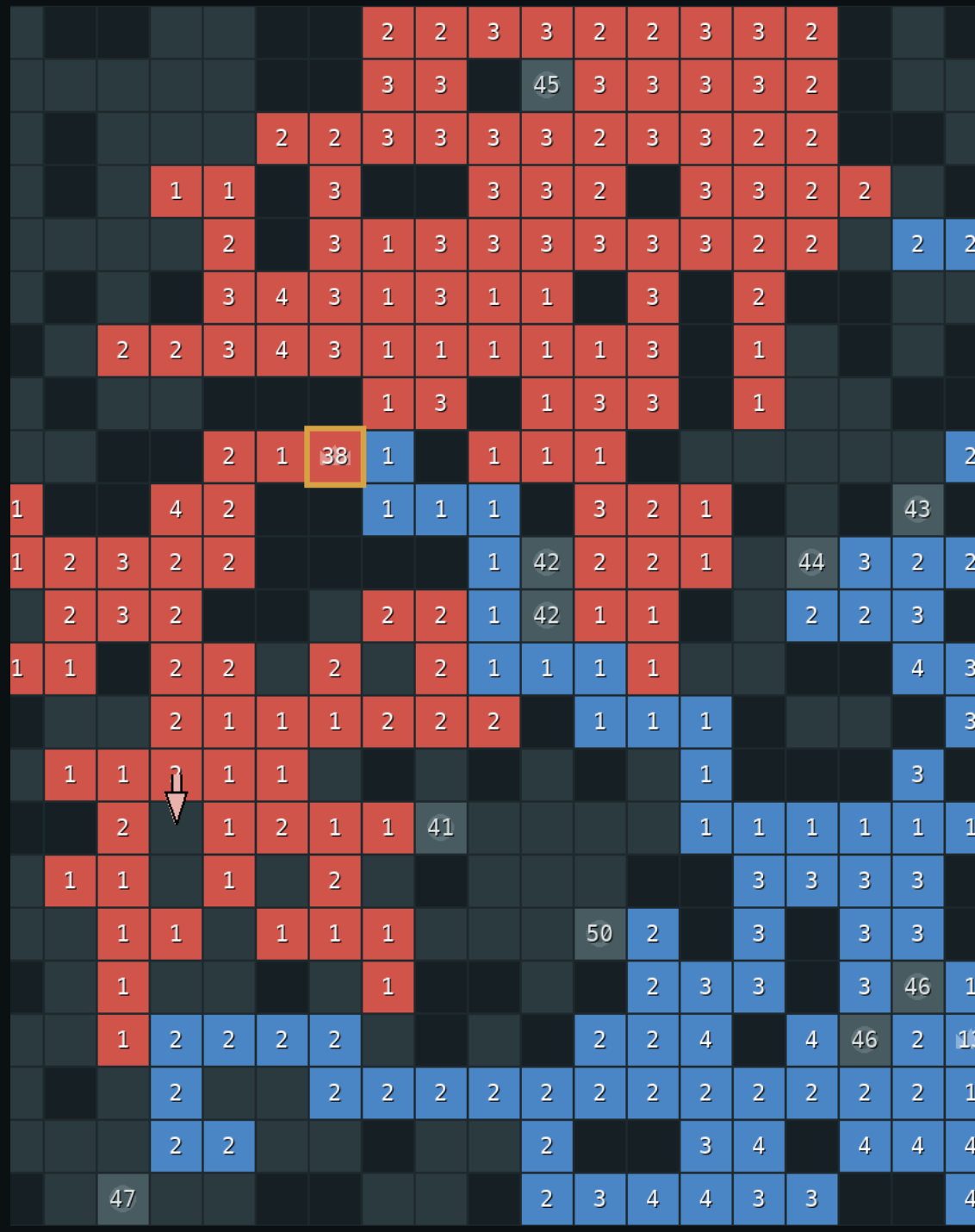
Until the opponent charges, both armies grow together.

The attack costs both sides; our general survives.

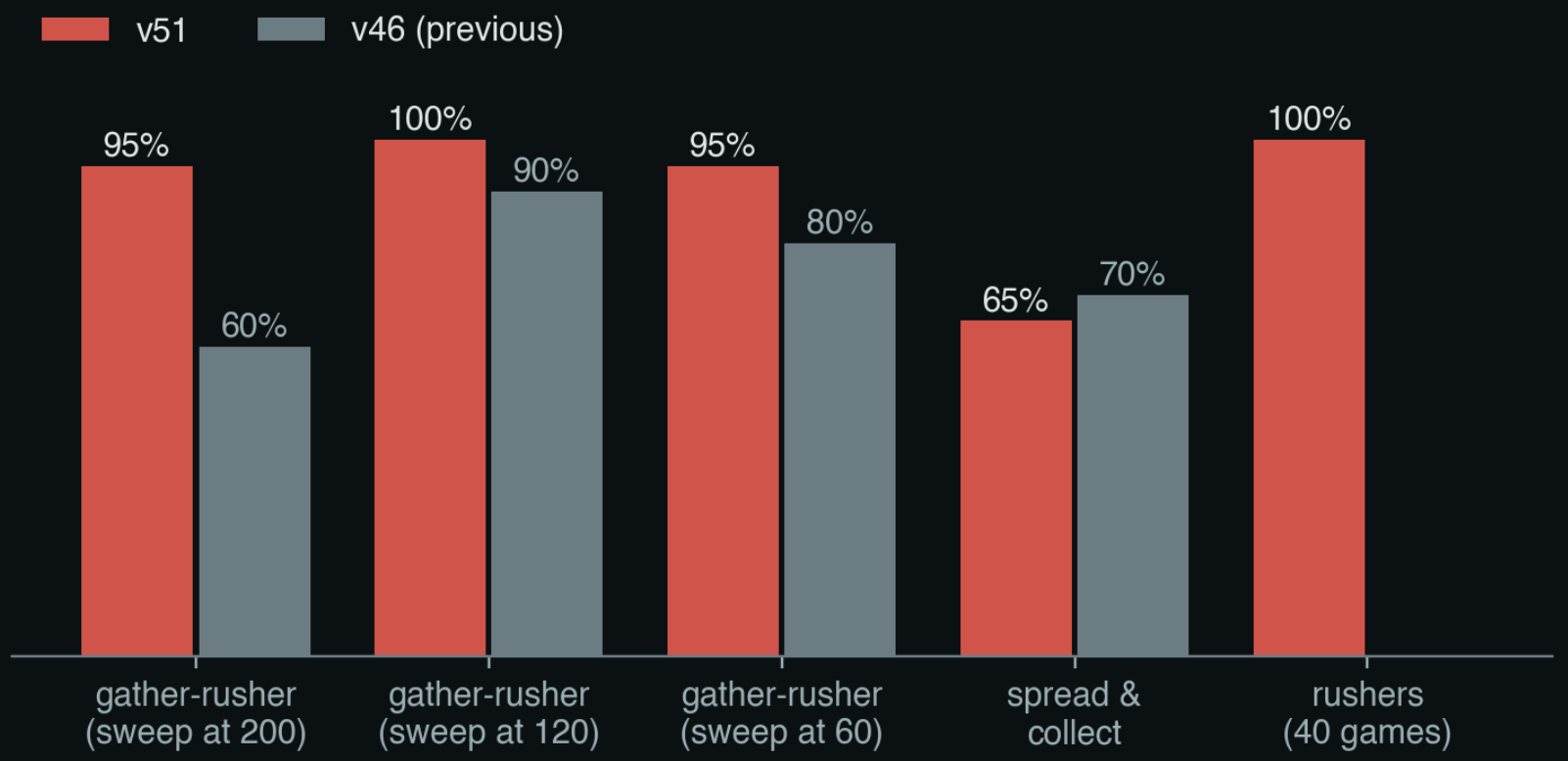
After the first castle, our bot pulls away until the capture.

5
Results, then you play

How it does against other bots, and what to try against it.



Where v51 is stronger



Bots that collect and charge: the biggest improvement.

Spread-and-castle player: slightly worse.

Against our previous bot: about even, 32–28.

20 games per bar. Grey bars: our previous bot.

Your turn

```
python play_arena.py   then pick  Final · champion
```

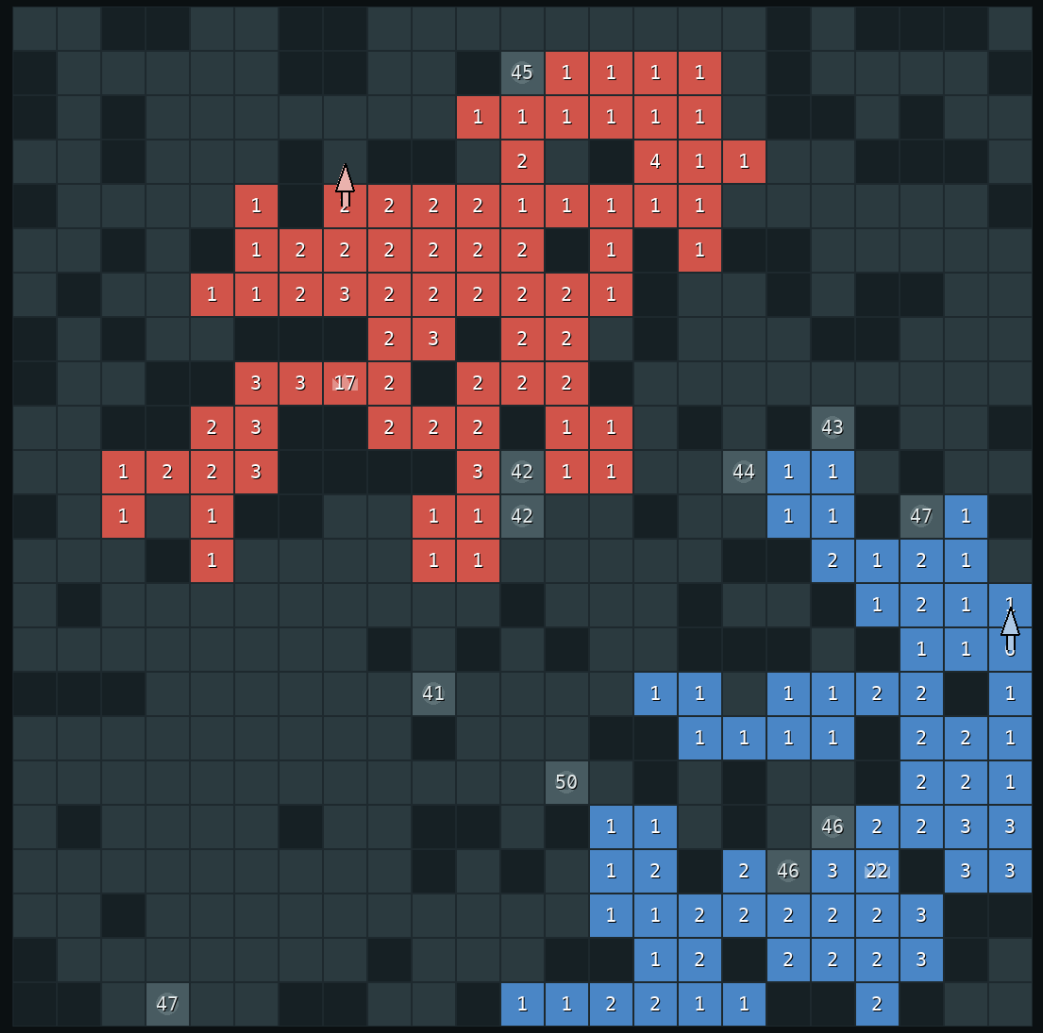
What to watch for:

Opening: it grabs land, one square per move.

Send a big stack at its general: it stops and pulls soldiers in.

Keep your general full: it won't attack.

To beat it: spread out and take castles.



REFERENCES

Book and papers

Russell, S. & Norvig, P. Artificial Intelligence: A Modern Approach, 4th ed. Pearson, 2020.

Straka, M., Lisý, V. & Schmid, M. Superhuman AI for Generals.io using self-play reinforcement learning. arXiv 2606.23348, 2026.

Barriga, N., Stanescu, M. & Buro, M. Puppet Search: enhancing scripted behavior by look-ahead search with applications to RTS games. AIIDE 2015.

Synnaeve, G. & Bessière, P. A Bayesian model for opening prediction in RTS games with application to StarCraft. AIIDE 2011.

Weber, B. & Mateas, M. A data mining approach to strategy prediction. CIG 2009.

Ganzfried, S. & Sandholm, T. Safe opponent exploitation. ACM Transactions on Economics and Computation, 2015.

Churchill, D. & Buro, M. Build order optimization in StarCraft. AIIDE 2011.

Started from (section 1): Likhachev, Gordon & Thrun, ARA*, NIPS 2003; Koenig & Likhachev, RTAA*, AAMAS 2006; Hernández & Baier, depression avoidance, AAAI 2012; Buro, ProbCut, ICCA J. 1995; Russell & Wefald, metareasoning, AIJ 1991.